NAZARBAYEV UNIVERSITY

# Programming for Engineers

## Detailed Reading Notes

## Week 10: Understanding Program Efficiency I

**Yanwei Wang**

School of Engineering

Nazarbayev University

Fall 2026

**Define the size. Count the work.**
**Classify growth. Measure. Verify again.**

SCHOOL OF ENGINEERING

# About these notes

These notes accompany the supplied 87-page presentation *Programming for Engineers: Week 10 — Understanding Program Efficiency I*. The slides determine the topic sequence, terminology, numerical examples, engineering framing, and optional scope. The notes expand their short statements into explanations, hand traces, derivations, and worked answers for independent study.

**Course information**

**Course:** ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS.
**Instructor:** Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
**Term:** Fall 2026.
**Week 10 scope:** Measuring computational work, orders of growth, Big-O notation, complexity classes, and characteristic examples. The source overview identifies CLOs 1 and 2.

*Lecture-slide alignment: Week 10 source PDF, pp. 1–3.*

## The three reading units

| Source lecture | Question developed in the notes |
|---|---|
| 28: Measuring computational work | What grows when an engineering problem gets larger, and how can the work be counted and timed responsibly? |
| 29: Orders of growth and Big-O | How can an exact count become a justified statement about growth without pretending to predict seconds? |
| 30: Recognizing complexity classes | Which execution structure explains each class, and how can repeated work be removed without changing the required result? |

Week 10 reuses familiar functions, loops, lists, recursion, tests, and objects. It does not require a new software framework. Deeper efficiency analysis remains in Week 11; the dedicated implementation of searching and sorting remains in Week 12. Optional material from the end of the presentation is retained in Chapter 4, not silently made a prerequisite.

## How to study this material

For each example, first identify the input, output, and valid domain. State what the size variable means. Trace a small input and count the *specified* action. Only then attach a complexity label. Before reading a worked answer, make a prediction on paper. Execute the companion program to inspect the result, and explain any difference between your prediction and the observation.

Do not judge your understanding by whether you can memorize a growth table. A useful explanation names the action, derives how often it occurs, identifies the input case, and states the assumptions that allow the count to represent computational work.

**Key idea**

**The week's workflow:** define the size → count the work → classify growth → measure → improve → verify again. Correctness, units, output meaning, and the numerical accuracy requirement remain part of the task throughout.

## Source examples, worked additions, and measurements

All displayed Python examples contain explanatory comments. The functions printed in the slides are retained with their teaching contracts; comments do not add an implicit validation pass. Where a slide shows only a function, small client calls have been added and identified as demonstrations. The complete earlier Week 10 source files, where available, were used to resolve line wrapping. Worked activity solutions and cumulative practice are labelled additions to the slide exposition.

The timing table in Chapter 1 is the *recorded experiment reported by the source slides*, not a new measurement on your computer or a guaranteed performance specification. Its earlier CSV and environment record are included unchanged. The mathematical growth plots are recreated from the stated formulas and are labelled models. Newly executed timing scripts produce local observations; they are not substituted for the source data.

**Two numbering systems**

Every source reference in these notes uses the **physical PDF page, from 1 to 87**. The slide footer numbers differ because lecture-divider pages are not counted in the same way. Appendix A maps every physical source page to a reading section.

## Running the examples

The numbered files in `examples/` run with Python's standard library. Run a file from that folder, for example `python 01_count_warnings.py`. Keep `reading_models.py` alongside the examples that import it. The separate `ENG101_Week10_Annotated_Examples.py` is self-contained and runs the principal deterministic demonstrations. The four timing examples are identified in the example index; their printed durations will vary.

The source ZIP includes all LaTeX chapters, logos, editable diagrams, data, examples, captured outputs, and verification tools. Compile the main document with pdfLaTeX or latexmk. The single-file LaTeX alternative embeds chapter text, listings, output, and plotted coordinates. Branding images are optional in that version, but retain the supplied assets for the same appearance.

The exercises are for reading and practice. No deadlines, weighting, or assessment rules are introduced here. Moodle remains the authority for separately announced requirements.

# Contents

**About these notes** 2

**1. Measuring Computational Work** 6
1.1. Why scale changes the question . . . 6
1.2. Define what the input size measures . . . 7
1.3. Choose a resource and a cost model . . . 7
1.4. A full scan: count every comparison . . . 8
1.5. Early exit and the required output . . . 9
1.6. An average needs a probability assumption . . . 10
1.7. Hidden scans in familiar expressions . . . 11
1.8. Measure a duration, not a clock reading . . . 11
1.9. Read the recorded source experiment . . . 13
1.10. Use scaling ratios as diagnostics . . . 14
1.11. Worked activity: a reproducible comparison . . . 14
1.12. A method call can hide work too . . . 16
1.13. Checklist and Lecture 28 exit ticket . . . 16

**2. Orders of Growth and Big-O** 18
2.1. From a count to a growth description . . . 18
2.2. Big-O is an eventual upper bound . . . 20
2.3. Work through a complete bound argument . . . 20
2.4. A tight order says more than an upper bound . . . 21
2.5. Choose the case, then bound its cost . . . 22
2.6. Worked simplification activity . . . 22
2.7. Consecutive stages add their work . . . 23
2.8. Independent nested dimensions multiply . . . 23
2.9. A fixed inner loop is not a second growing dimension . . . 24
2.10. Dependent bounds require summation . . . 25
2.11. A hidden membership scan adds work . . . 26
2.12. Conditionals select routes, not both branches . . . 26
2.13. Constants still matter at practical sizes . . . 27
2.14. Logarithm bases and exact counts . . . 28
2.15. Debug the explanation and complete the exit ticket . . . 28

**3. Recognizing Complexity Classes** 29
3.1. Attach each class to a reason . . . 29
3.2. Constant work: one known position . . . 30
3.3. Logarithmic work: repeatedly halve the size . . . 31
3.4. Interval refinement is another halving process . . . 31
3.5. Linear work: integrate a sampled power record . . . 32
3.6. Log-linear work: reset the stages for each item . . . 33
3.7. Quadratic work: compare every ordered pair . . . 34
3.8. Cubic work and the choice of dimension . . . 35
3.9. Exponential branching: count leaves and calls . . . 36
3.10. Fibonacci exposes repeated subproblems . . . 37
3.11. Recursion is not itself a complexity class . . . 39
3.12. Compare normalized growth models . . . 39
3.13. Worked activity: a normalized work budget . . . 40

3.14. Separate time, output, and additional state . . . 41
3.15. Integrated improvement: all cumulative totals . . . 41
3.16. Correct the quick-poll misconceptions . . . 43
3.17. Lecture 30 exit ticket and synthesis . . . 44

**4. Optional Reference and Preparation** **45**
4.1. Exact counts include the final condition test . . . 45
4.2. A standard helper for microbenchmarks . . . 46
4.3. Integer magnitude and representation length . . . 46
4.4. Reference costs for familiar operations . . . 47
4.5. An optional factorial-growth contrast . . . 47
4.6. Worked preparation for Week 11 . . . 48
4.7. Where the course goes next . . . 49

**5. Cumulative Practice with Worked Solutions** **50**
5.1. P1: output events versus executed comparisons . . . 50
5.2. P2: preserve an early-exit contract . . . 50
5.3. P3: justify the average-case assumption . . . 50
5.4. P4: compute the selected three-stage count . . . 51
5.5. P5: supply an actual Big-O witness . . . 51
5.6. P6: keep the dimensions explicit . . . 51
5.7. P7: compare pair definitions . . . 51
5.8. P8: distinguish a floor from a ceiling . . . 52
5.9. P9: count recursive work rather than output magnitude . . . 52
5.10. P10: read a batch timing correctly . . . 52
5.11. P11: preserve all requested prefix results . . . 52
5.12. P12: judge the evidence in an efficiency report . . . 53
5.13. A final readiness check . . . 53

**6. Quick Reference and Glossary** **54**
6.1. The compact analysis procedure . . . 54
6.2. Terminology . . . 54

**A. Source Basis and Slide Coverage** **57**
A.1. What determines these notes . . . 57
A.2. Reproducibility and package contents . . . 58
A.3. Map from every source page to these notes . . . 58

# 1 Measuring Computational Work

**Reading route:** Lecture 28; source PDF pages 4–28.

**Learning outcomes**

After studying this chapter, you should be able to define input size and a selected operation, separate work from elapsed time and extra storage, trace full scans and early exits, distinguish input cases, plan a repeatable timing experiment, and describe exactly what its evidence supports.

*Lecture-slide alignment: Week 10 source PDF, pp. 4–5.*

## 1.1. Why scale changes the question

A correct program for ten measurements may become impractical for a million measurements. The mathematical rule has not changed, but the required amount of work, the storage, or the deadline may have changed. A prototype can therefore hide a difficulty that appears only at the intended application size.

Consider three expressions applied to an already-created list of finite numerical readings:

| Expression | Work described in the source warm-up |
|---|---|
| `readings[-1]` | Retrieve a value at one known position. |
| `sum(readings)` | Visit the readings to form a total. |
| `100.0 in readings` | Inspect entries until a match is found or the list is exhausted. |

Each expression fits on one line, but a line of source code is not a fixed unit of execution. List indexing can retain a fixed amount of work when the list grows. Summation cannot obtain the complete total without processing the numerical input. Membership may finish early, but an absent target requires every candidate to be ruled out.

**Warm-up answer**

For a nonempty existing list, `readings[-1]` is the fixed-work choice under the stated list-access model. For membership, we still need to know whether the target occurs and where the first occurrence lies. A million-element list does not itself tell us which path a membership test will take.

Two programs can produce the required result and still have different scaling. A one-pass summary grows with the number of readings; an all-pairs comparison grows much faster. However, a full summary and a first-match search also have *different output requirements*. We must not claim an improvement by replacing the requested output with a cheaper but incomplete answer.

*Lecture-slide alignment: Week 10 source PDF, pp. 6–7.*

## 1.2. Define what the input size measures

The symbol $n$ has no universal meaning. It is a variable selected to describe the task. The first sentence of an efficiency claim should make that choice explicit.

| Task | Appropriate size description |
|---|---|
| Process a temperature log | $n$: number of readings. |
| Compare two collections | $n$ and $m$: separate collection lengths. |
| Visit a square grid | $q$: points per side; alternatively $N = q^2$: total points. |
| Refine a bracket numerically | $L_0/\varepsilon_x$: initial width divided by required final width. |

Suppose a calculation visits all points of a square grid once. It visits $q^2$ locations when there are $q$ points on each side. The same count is $N$ when $N$ denotes the total number of locations. Saying "quadratic" in the first description and "linear" in the second is not a contradiction: the independent variable has changed.

Likewise, $n$ diameter choices and $m$ speed choices produce $nm$ candidate pairs. Replacing $m$ with $n$ is justified only when the problem actually specifies that both axes grow together. A constant number of speeds is another distinct case.

For interval refinement, the ratio $L_0/\varepsilon_x$ is dimensionless when both widths have the same unit. A required width of 0.125 s has a different meaning from a residual tolerance in metres. The source keeps those questions separate, and so will we.

*Lecture-slide alignment: Week 10 source PDF, p. 8.*

## 1.3. Choose a resource and a cost model

### 1.3.1. Work, time, and storage are different quantities

**Time complexity** describes the growth of executed work in a chosen model. A **measured duration** describes one implementation, input, machine, and environment. **Auxiliary space** describes additional storage beyond whatever the analysis explicitly excludes.

A running sum reads all $n$ values but needs only a fixed number of running scalars. A function that returns $n$ converted readings must also retain those output values. These methods may have the same linear work order but different storage requirements. Neither description replaces the other.

Throughout the core examples, assume list indexing, bounded-cost comparisons, and arithmetic on bounded-size numbers cost a constant amount per execution. Count the number of executions of a chosen operation. Include work inside called functions and methods. These assumptions make a useful introductory model; they do not claim that every Python statement takes one CPU instruction.

**Common mistake**

"There is one operation on this line" is not a sufficient analysis. Copying an $n$-element list can copy $n$ references. Comparing long strings may inspect multiple characters. Arithmetic on increasingly large Python integers can require increasing work and storage.

### 1.3.2. State exactly what is counted

In a threshold monitor, we can count threshold comparisons, warning events, loop-condition tests, or assignments. All are legitimate observations, but their numerical values need not match. An *exact selected-operation count* is exact only for the operation that has been named. It is not automatically the exact total work of the program.

We will use $C(n)$ for a selected count and $T(n)$ for a general cost or time model when useful. In the formal Big-O definition, the symbol $C$ is also traditionally used for a fixed multiplier. Context distinguishes that constant from the count function $C(n)$.

*Lecture-slide alignment: Week 10 source PDF, pp. 9–11.*

## 1.4. A full scan: count every comparison

The first program returns two quantities: how many temperatures exceed a limit and how many threshold comparisons were performed. Its input contract assumes finite numerical readings and a finite threshold. It does not scan again to validate that assumption.

```python
def count_warnings(readings_c, limit_c):
    """Return (warning count, threshold tests); inputs are finite readings."""
    warnings = 0  # Output: how many values exceed the limit.
    checks = 0  # Work counter: how many values are inspected.
    for value_c in readings_c:
        checks += 1  # Count a comparison even if it finds no warning.
        if value_c > limit_c:  # Equality is permitted by this rule.
            warnings += 1
    return warnings, checks  # An empty input returns (0, 0).

# Source example: 31 and 35 exceed 30 deg C; all four are inspected.
print(count_warnings([19.0, 30.0, 31.0, 35.0], 30.0))
print(count_warnings([], 30.0))  # No threshold test is performed.
```

**Code 1.1:** Warning events and comparison work are counted separately.

```
(2, 4)
(0, 0)
```

The strict comparison `value_c > limit_c` makes exactly 30 °C acceptable under this particular teaching rule. The counter `checks` is incremented for every reading, before the decision; `warnings` changes only on the true branch. Moving both increments into the true branch would lose the work information.

| Reading (°C) | Above 30? | Warnings | Checks |
|---:|---|---:|---:|
| 19.0 | False | 0 | 1 |
| 30.0 | False | 0 | 2 |
| 31.0 | True | 1 | 3 |
| 35.0 | True | 2 | 4 |

For any valid list of length $n$, exactly $n$ threshold comparisons execute:

$$C_{\text{checks}}(n) = n, \qquad 0 \le C_{\text{warnings}} \le n.$$

A zero-warning log is not a zero-work log. We still inspect each reading to establish that no warning exists. A function required to count *all* warnings cannot stop at the first one.

### 1.4.1. Empty and boundary cases

The empty list returns `(0, 0)`; `[30.0]` returns `(0, 1)`; `[31.0]` returns `(1, 1)`. These cases verify both the decision boundary and the distinction between output and work.

At $n = 0$ the selected comparison count is zero. The function call, scalar initialization, and return still happen. Consequently "zero comparisons" does not mean "zero seconds." This is the first example of why a selected count must not be confused with complete runtime.

*Lecture-slide alignment: Week 10 source PDF, pp. 12–14.*

## 1.5. Early exit and the required output

The next function asks a different question: where is the *first* temperature strictly above the limit? Once that index is known, later readings cannot change which index was first. Early return is now correct.

```
1  def first_warning(readings_c, limit_c):
2      """Return (first warning index or None, number of threshold tests)."""
3      checks = 0  # Count the successful comparison as well as failures.
4      for index, value_c in enumerate(readings_c):
5          checks += 1
6          if value_c > limit_c:
7              return index, checks  # Stop: the required first index is known.
8      return None, checks  # No match, including the empty-input case.
9
10 # Added demonstration calls illustrating the source's execution cases.
11 print(first_warning([19.0, 30.0, 31.0, 35.0], 30.0))
12 print(first_warning([31.0, 19.0, 20.0, 22.0], 30.0))
13 print(first_warning([19.0, 20.0, 22.0, 30.0], 30.0))
14 print(first_warning([], 30.0))
```

**Code 1.2:** First warning index and work up to that result.

The function follows the source listing. The four demonstration calls are added to illustrate the source's execution cases.

```
(2, 3)
(0, 1)
(None, 4)
(None, 0)
```

The first output `(2, 3)` means that index 2 is the first warning and three values were inspected. Indexing begins at zero, but the successful third comparison still counts. The second output shows a match at index zero after one comparison. The third shows that absence requires the full four-element scan. The empty input performs no threshold tests.

| Case for $n \geq 1$ | Threshold comparisons |
|---|---|
| First reading warns | 1 |
| First warning at index $k$ | $k+1$ |
| Only the final reading warns | $n$ |
| No warning exists | $n$ |

**Best-case work** is the smallest work among valid inputs of the same size. **Worst-case work** is the largest. The size is held fixed while the values or their arrangement change. For this function and $n \geq 1$, the best comparison count is 1 and the worst is $n$.

**Engineering check**

The first-warning program is not a drop-in replacement for a full warning count. It returns a location, not a total. Efficiency comparisons must retain the requested result, units, interpretation, and failure behavior.

*Lecture-slide alignment: Week 10 source PDF, pp. 15–16.*

## 1.6. An average needs a probability assumption

The slides deliberately use a restricted model: assume a warning exists, and its *first* position is equally likely to be any of the $n$ positions. The possible comparison counts are then $1, 2, \ldots, n$, each with probability $1/n$.

Using the arithmetic-series identity,

$$C_{\text{avg}}(n) = \frac{1+2+\cdots+n}{n} = \frac{n(n+1)/2}{n} = \frac{n+1}{2}.$$

For $n = 4$, this gives 2.5 comparisons on average. A single run cannot execute half a comparison. The fractional value is an expectation across the specified possible inputs.

Real sensor records may have no warning, clustered events, or a nonuniform first-warning position. The formula must not be silently applied to them. An average case is not obtained by averaging the best and worst counts *without a distribution*; the expression above works because the stated uniform model makes that arithmetic justified.

**Stop, predict, and try it**

Under the source's conditional uniform-position assumption, predict the mean number of checks for $n = 9$. Then explain why the answer is not a guaranteed number of checks for every nine-reading log.

**Answer and explanation**

The expectation is $(9+1)/2 = 5$. Individual logs still require from 1 through 9 checks when a warning exists; an absent warning also requires 9. The value 5 describes only the specified probability model.

*Lecture-slide alignment: Week 10 source PDF, p. 17.*

## 1.7. Hidden scans in familiar expressions

```python
# Four finite numeric values; constructing the list is separate work.
readings = [2.0, 4.0, 6.0, 8.0]
print(len(readings))  # Read the stored length of this existing list.
print(readings[-1])  # Retrieve one known position.
print(sum(readings))  # Visit every numerical element to form a total.
print(9.0 in readings)  # An absent target requires the full search.
```

**Code 1.3:** Inspect the meaning of a call rather than its source length.

```
4
8.0
20.0
False
```

For this existing Python list, `len()` obtains its stored length, and `[-1]` retrieves a known element. Neither requires another traversal of the four values in the classroom model. In contrast, `sum()` processes them, and the absent membership target must be checked against every candidate.

The printed result is not a work counter: `20.0` is the sum, not the number of operations. Likewise, `False` does not indicate whether one or a million unsuccessful comparisons were needed. Reading the implementation or the operation's specified behavior is necessary to explain cost.

*Lecture-slide alignment: Week 10 source PDF, p. 18.*

## 1.8. Measure a duration, not a clock reading

```python
# Import an elapsed-time counter from the standard library.
from time import perf_counter

values = [1.0] * 100_000  # Build the input before starting the timer.
start_s = perf_counter()  # Record the earlier counter reading.
result = sum(values)  # This is the calculation being measured.
elapsed_s = perf_counter() - start_s  # Difference, in seconds.
assert result == 100_000.0  # Check the answer outside the timed region.
print(f"Elapsed: {elapsed_s:.6f} s")  # Your measured duration will vary.
```

**Code 1.4:** An elapsed-time observation for an already-created input.

The two calls to `perf_counter()` bracket the calculation. Their difference is the observed duration in seconds. The individual counter values are not the answer of interest. The input list is constructed before timing; the assertion and printing occur afterward.

This separation gives a *calculation-only* experiment. It is not a measurement of the whole application from input construction to displayed report. Those excluded activities still take time in an end-to-end workflow. A different engineering question may require measuring them too.

**Evidence to report**

This code prints a machine-dependent value in the form `Elapsed: ... s`. No fixed elapsed value is supplied as a reference answer. Timer overhead, other activity, and implementation details can influence the observation, especially for short calculations.

### 1.8.1. Repeat before making a claim

```python
# Use the same task and input for all five timing trials.
from time import perf_counter

values = [1.0] * 100_000  # Setup is outside the measured region.
sum(values)  # One warm-up, not a recorded trial.
durations_s = []  # Retain raw observations rather than just one summary.
for trial in range(5):
    start_s = perf_counter()
    result = sum(values)
    durations_s.append(perf_counter() - start_s)
    assert result == 100_000.0  # Correctness is checked after timing.
print("Trial times (s):", durations_s)
print("Best observed (s):", min(durations_s))  # Not a deadline guarantee.
```

**Code 1.5:** Keep all five observations and name the selected summary.

The warm-up uses the same task but is not recorded as one of the five trials. The list `durations_s` preserves each observation, allowing another person to inspect variability. The minimum is the *best observed trial*; it is not a worst-case guarantee, a deadline promise, or an average. A median describes the middle observed duration and answers a different descriptive question.

Correctness checks belong outside the timed region when the claim concerns only the calculation. They must still be performed. Excluding them from a timer does not make them optional.

### 1.8.2. A fair comparison

Keep the required output and equivalent input data fixed. Use the same environment for comparisons and describe the timed region. Repeat at several sizes. Avoid including printing or plotting on only one side. When a method mutates its input, restore equivalent starting data before the next trial; otherwise a later call may receive an easier or different problem.

**Common mistake**

Comparing a fresh dataset against a dataset already modified by the first method can produce a misleading result. So can timing setup for one implementation but not the other. State exclusions, rather than hiding them behind the word “faster.”

*Lecture-slide alignment: Week 10 source PDF, pp. 19–21.*

## 1.9. Read the recorded source experiment

The presentation reports a summation experiment using $n$ copies of `1.0`. It records one warm-up followed by five batches of 20 calls for each size. Setup, correctness checks, and output are excluded. The recorded environment is CPython 3.13.5 on Linux x86_64.

| $n$ | Best (ms/call) | Median (ms/call) | Trials × calls |
|---:|---:|---:|---:|
| 1,000 | 0.0069 | 0.0069 | $5 \times 20$ |
| 10,000 | 0.0698 | 0.0713 | $5 \times 20$ |
| 100,000 | 0.7008 | 0.7134 | $5 \times 20$ |
| 1,000,000 | 7.2199 | 7.4085 | $5 \times 20$ |

Each raw observation is a *batch* duration. Dividing by 20 converts it to a per-call value; multiplying seconds by 1000 converts it to milliseconds. The table is rounded exactly as in the slides. The complete earlier CSV and environment file are included unchanged as `data/timing.csv` and `data/timing_environment.json`.

**Provenance of these numbers**

These are the source presentation's recorded measurements, recovered from its earlier data files. They are not newly generated evidence and are not a required outcome for students. The batch records describe variation within that original run, not performance on all computers.

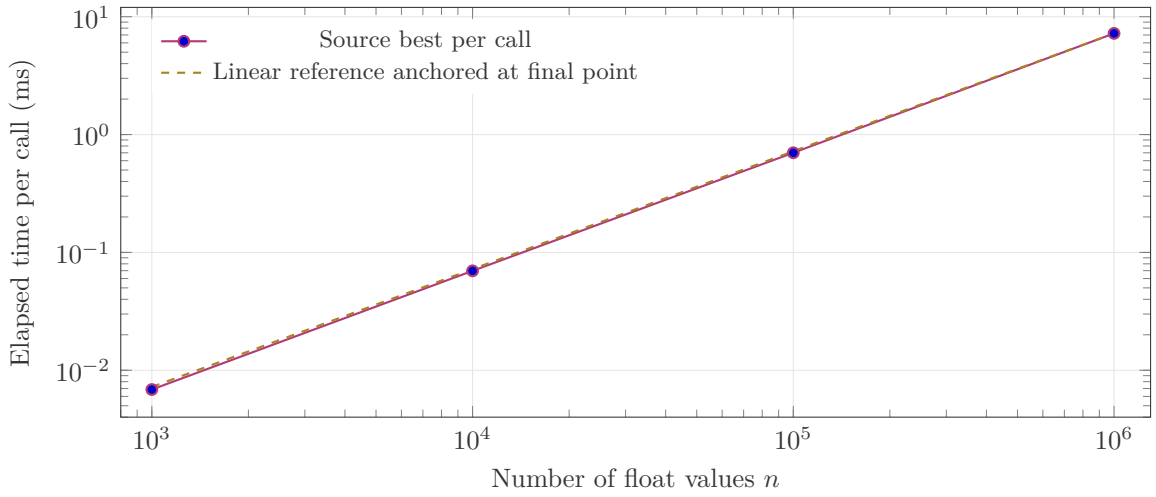



**Figure 1.1.:** Recreated source-page-23 timing graph, using the recovered original CSV. Both axes are logarithmic. The reference line is proportional to $n$ and anchored at the final measured best-per-call point; it is not an independent measurement or a fitted proof.

On logarithmic axes, equal horizontal distances correspond to equal multiplicative changes in input size. A proportional model $t(n) = an$ satisfies $\log t = \log a + \log n$, so it appears as a straight line of slope one on a log–log plot with corresponding scales. The plotted measurements broadly follow that reference over the tested range. Together with the scan argument, they support the reported linear trend, but finite measurements do not prove all-size asymptotic behavior.

*Lecture-slide alignment: Week 10 source PDF, pp. 22–23.*

## 1.10. Use scaling ratios as diagnostics

Define the doubling ratio

$$R(n) = \frac{T(2n)}{T(n)}.$$

For the exact constant model $T(n) = a > 0$, the ratio is 1. For $T(n) = an + b$ with $a > 0$,

$$R(n) = \frac{2an + b}{an + b} \longrightarrow 2.$$

For a positive-leading quadratic model $T(n) = an^2+bn+c$, division of numerator and denominator by $n^2$ shows that the ratio approaches 4. These statements describe dominant models, not a guarantee for every pair of stopwatch observations.

The source timing table increases $n$ by a factor of ten, not two. It therefore does *not* directly supply the displayed doubling ratios. If we compare adjacent rows, the linear reference suggests a factor near ten. A separate experiment at $n$ and $2n$ would be needed to observe a doubling ratio directly.

Small-input overhead, measurement noise, cache effects, and setup can distort ratios. Use ratios to diagnose whether a scaling explanation is plausible, alongside the code and its count, rather than to assign a complexity class from one noisy observation.

*Lecture-slide alignment: Week 10 source PDF, p. 24.*

## 1.11. Worked activity: a reproducible comparison

The slide activity asks you to compare `sum(values)` with a Python loop accumulating every value. One possible design uses list lengths $10^3$, $10^4$, and $10^5$, all filled with `1.0`. The expected sum is then independently known to be $n$.

```python
# Added solution to the slide activity, not the source benchmark data.
from statistics import median
from time import perf_counter

def loop_total(values):
    """Return a total by explicitly accumulating each supplied value."""
    total = 0.0
    for value in values:
        total += value  # One accumulation for each numerical element.
    return total

for n in [1_000, 10_000, 100_000]:
    values = [1.0] * n  # Same already-created input for both methods.
    expected = float(n)  # Independent answer for a list of ones.
    for method in [sum, loop_total]:
        assert method(values) == expected  # Also serves as a warm-up.
        trials = []
        for trial in range(5):
            start = perf_counter()
            result = method(values)  # Time only this call.
            trials.append(perf_counter() - start)
            assert result == expected  # Check after stopping the timer.
        print(n, method.__name__, min(trials), median(trials))
        # Keep raw trials so the summary can be checked.
        print("Raw seconds:", trials)
```

**Code 1.6:** Added worked solution to the measurement-design activity.

This solution checks the result before timing, uses the same input for both methods, captures five trials per method and size, and reports both the minimum and median while retaining all observations. It is not the batching implementation behind the recorded source table: it times one call per trial and compares two methods. That difference is intentional and must be disclosed.

Both methods traverse the input in their algorithmic description. They may nevertheless have different measured constants because their implementations differ. This activity does not establish that an explicit Python loop is always preferable or always unacceptable. Report the actual observations and the tested conditions.

*Lecture-slide alignment: Week 10 source PDF, p. 25.*

## 1.12. A method call can hide work too

```python
class SensorLog:
    """Store a separate outer list; assume finite numeric readings."""

    def __init__(self, readings_c):
        self.readings_c = readings_c.copy()  # Copy n references.

    def count_high(self, limit_c):
        count = 0  # Only this scalar is needed during the scan.
        for value_c in self.readings_c:  # Every call inspects n readings.
            if value_c > limit_c:
                count += 1
        return count

# Added client calls: initialization and querying are separate costs.
raw = [19.0, 30.0, 31.0, 35.0]
log = SensorLog(raw)
print(log.count_high(30.0))
print(raw)  # The constructor copied rather than replacing this input.
```

**Code 1.7:** Construction copies references; each query scans the stored list.

The class follows source page 26; the short client is added for direct execution.

```
2
[19.0, 30.0, 31.0, 35.0]
```

The constructor's `copy()` creates a new outer list of $n$ references. This takes work and retained storage that grow with $n$. For the flat numerical data used here, it also prevents later changes to the caller's *outer list* from directly changing the stored sequence.

Each `count_high()` query performs $n$ threshold tests. The method itself retains only a running counter beyond the already-stored readings. Calling it through an object does not change the underlying scan into fixed work. A useful interface hides implementation detail for the caller; it does not erase that implementation's computational cost.

*Lecture-slide alignment: Week 10 source PDF, p. 26.*

## 1.13. Checklist and Lecture 28 exit ticket

Before making an efficiency claim, answer six questions: what does $n$ represent; what output and input contract are required; which operations are counted; which input case is analyzed; what setup or storage is excluded; and what did measurement actually establish?

A stronger statement than "it is fast" is: *For $n$ finite readings, this function compares every reading once with the same limit and keeps only a fixed number of running scalars, excluding input storage.* This supplies a checkable explanation even before any timer is used.

**Source exit ticket**

A pressure log contains $n$ readings. A function returns the first reading strictly above 200 kPa, or `None`. Give its best and worst threshold-comparison counts for $n \geq 1$. Explain why three source lines need not mean fixed work. Name one quantity held fixed in a timing comparison.

**Answer and explanation**

The best count is 1 when the first reading exceeds the threshold. The worst is $n$ when the first warning is last or there is no warning. A short function can contain a loop or invoke a scan. Hold the required result and input data fixed; the environment and measured region must also be comparable. This exercise asks for the *reading value*, whereas the earlier temperature function returns an *index*.

```python
# Added answer to Lecture 28's pressure exercise.
def first_high_pressure(pressures_kpa):
    """Return (first value above 200 kPa or None, threshold tests)."""
    checks = 0
    for value_kpa in pressures_kpa:
        checks += 1  # Include the first successful comparison.
        if value_kpa > 200.0:
            return value_kpa, checks  # Exercise asks for the value.
    return None, checks

print(first_high_pressure([100.0, 200.0, 210.0, 300.0]))
print(first_high_pressure([100.0, 200.0]))  # Equality is not a warning.
```

**Code 1.8:** Added executable answer retaining the exit ticket's value-returning contract.

```
(210.0, 3)
(None, 2)
```

*Lecture-slide alignment: Week 10 source PDF, pp. 27–28.*

# 2 Orders of Growth and Big-O

**Reading route:** Lecture 29; source PDF pages 29–53.

**Learning outcomes**

Separate an exact selected-operation count from its order of growth. Explain Big-O as an eventual upper bound. Distinguish a tight bound from an input case. Combine consecutive, nested, and conditional costs. Preserve independent size variables, and keep finite-size constants visible.

*Lecture-slide alignment: Week 10 source PDF, pp. 29–30.*

## 2.1. From a count to a growth description

An exact count is tied to the operation being counted. A growth description deliberately compresses that information: it identifies the behavior that controls the cost as the chosen size becomes sufficiently large.

The source warm-up separates three contributions:

$$A(n) = 1000, \qquad B(n) = n, \qquad C(n) = 2n^2.$$

| $n$ | **Fixed** 1000 | **Linear** $n$ | **Quadratic** $2n^2$ |
|---:|---:|---:|---:|
| 10 | 1,000 | 10 | 200 |
| 100 | 1,000 | 100 | 20,000 |
| 1,000 | 1,000 | 1,000 | 2,000,000 |
| 10,000 | 1,000 | 10,000 | 200,000,000 |

The final row is the worked answer to the next-row prediction. The fixed part does not disappear, but it becomes a smaller fraction of a rapidly growing total. The quadratic contribution eventually dominates both the fixed and linear terms.

*Lecture-slide alignment: Week 10 source PDF, p. 31.*

### 2.1.1. A program with three counted stages

```python
def selected_additions(n):
    """Count only count += 1 executions; assume integer n >= 0."""
    count = 0  # Initialization is not one of the selected additions.
    for _ in range(1000):
        count += 1  # Fixed stage: 1000 selected additions.
    for _ in range(n):
        count += 1  # Linear stage: n selected additions.
    for i in range(n):
        for j in range(n):
            count += 1  # First selected addition for this pair.
            count += 1  # Second: two additions per pair, n*n pairs.
    return count

print(selected_additions(10))  # 1000 + 10 + 2*100 = 1210.
```

**Code 2.1:** A fixed stage, a linear stage, and two additions per pair.

```
1210
```

Count only executions of `count += 1`. The first loop performs 1000, the second performs $n$, and the nested loops visit $n^2$ pairs with two such additions per pair. The total is

$$C(n) = 1000 + n + 2n^2.$$

For $n = 10$, the return value is 1210. The function entry, loop machinery, assignments to indices, and return are not included in this *selected* count. We would have a different exact formula if we counted those actions too.

The underscore `_` is used for an unused loop value in the first two stages. It communicates that the repeated action matters but the index does not. It does not remove the loop or reduce the number of executions.

*Lecture-slide alignment: Week 10 source PDF, p. 32.*

### 2.1.2. Dominance does not mean deletion

The source summarizes $1000 + n + 2n^2$ as quadratic, $7n + 30$ as linear, and the fixed cost 12 as constant. All terms still execute. The point is that fixed multiples and lower-order terms do not change the dominant growth class.

For the first count, dividing by $n^2$ makes the idea visible:

$$\frac{1000 + n + 2n^2}{n^2} = \frac{1000}{n^2} + \frac{1}{n} + 2.$$

The first two terms shrink as $n$ grows, leaving a quantity approaching 2. This is an explanatory derivation of the source's dominant-term statement, not a replacement of the actual work with only two instructions.

**Key idea**

An order of growth is not a stopwatch reading. It does not specify seconds, prove correctness, or select the fastest implementation at every small input size. Keep the growth argument and measured performance as different forms of evidence.

*Lecture-slide alignment: Week 10 source PDF, pp. 33–34.*

## 2.2. Big-O is an eventual upper bound

**Definition**

For an eventually nonnegative comparison function $g(n)$, the statement

$$T(n) \in O(g(n))$$

means that there exist fixed constants $C > 0$ and $n_0$ such that

$$0 \leq T(n) \leq Cg(n) \qquad \text{for every } n \geq n_0.$$

The constants may be chosen for this cost function but must not change as $n$ grows.

Read the definition in words: after some starting size, a constant multiple of $g(n)$ bounds the cost from above. The symbol $\in$ emphasizes that $O(g(n))$ is a family of functions satisfying this relation. The commonly written shorthand $T(n) = O(g(n))$ is not an ordinary equality giving an exact formula for $T(n)$.

The constant $C$ adjusts vertical scale. The threshold $n_0$ allows small-input exceptions. We need both to be independent of $n$. Choosing a new, larger multiplier separately for each input size would not establish the stated uniform bound.

*Lecture-slide alignment: Week 10 source PDF, p. 35.*

## 2.3. Work through a complete bound argument

Use the source example

$$T(n) = 3n + 20.$$

For every $n \geq 20$, we have $20 \leq n$, so

$$3n + 20 \leq 3n + n = 4n.$$

Therefore $C = 4$ and $n_0 = 20$ are explicit witnesses for $T(n) \in O(n)$.

At $n = 10$, $T(n) = 50$ is larger than $4n = 40$. This does not invalidate the proof, because the chosen threshold is 20. At $n = 20$, both expressions are 80; beyond it, the chosen upper line stays above the cost line.

Checking five values can illustrate this inequality. It cannot establish that an untested sixth or much larger value will satisfy it. The algebra provides the all-$n \geq 20$ argument. The graph helps explain the argument but is not a substitute for it.

**Stop, predict, and try it**

Why is the pair $C = 4$, $n_0 = 20$ more informative than simply writing “the graph looks linear”?

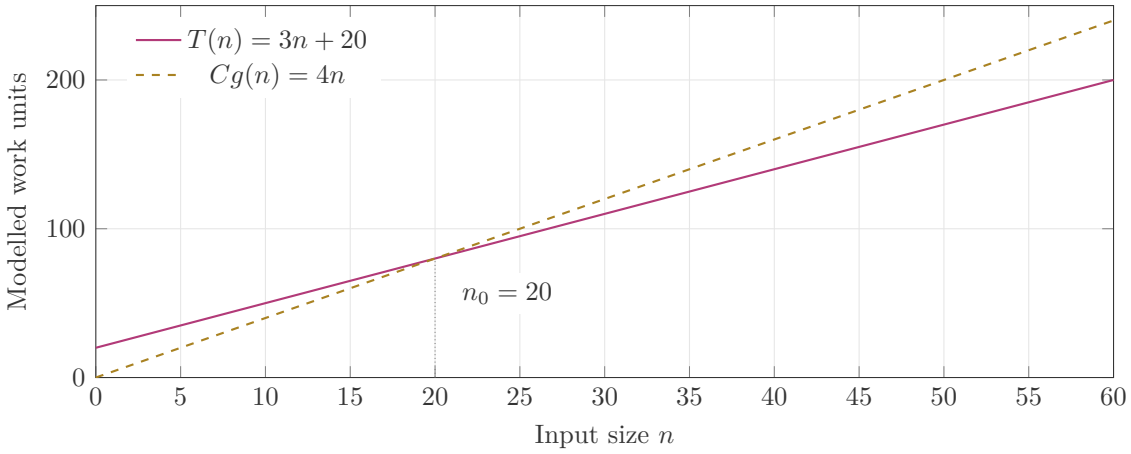



**Figure 2.1.:** The source-page-37 bound redrawn from its equations. The vertical axis is modelled work. With $C = 4$ and $n_0 = 20$, $3n + 20 \leq 4n$ for every $n \geq 20$. Failure of this particular inequality below the threshold is allowed.

**Answer and explanation**

The pair states a concrete inequality and the complete range over which it holds. The proof explains why every input in that range satisfies it. A visual resemblance over a plotted interval does not supply that guarantee.

*Lecture-slide alignment: Week 10 source PDF, pp. 36–37.*

## 2.4. A tight order says more than an upper bound

Big-O alone supplies an upper bound. A **tight order**, written $\Theta(g(n))$, supplies matching upper and lower bounds up to positive fixed multipliers for sufficiently large $n$. Thus $T(n) \in \Theta(n)$ means that the cost is eventually bounded both above and below by positive multiples of $n$.

For the same source example, $3n + 20 \geq 3n$ for nonnegative $n$, and the upper bound $3n + 20 \leq 4n$ holds for $n \geq 20$. Together they justify

$$3n \leq T(n) \leq 4n \quad (n \geq 20), \qquad T(n) \in \Theta(n).$$

It is also true that $T(n) \in O(n^2)$: for $n \geq 20$, $T(n) \leq 4n \leq 4n^2$. That bound is valid but loose; it describes less precisely what the function actually does.

**Key idea**

The classroom convention is to give the most informative simple bound justified by the count. Use $\Theta$ when explicitly asserting a tight order. Do not treat every valid Big-O bound as equally informative.

*Lecture-slide alignment: Week 10 source PDF, p. 38.*

## 2.5. Choose the case, then bound its cost

There are two distinct decisions. First select the cost function: best-case work, worst-case work, or expected work under stated input probabilities. Then describe that chosen function with an appropriate bound.

For the first-warning function and $n \geq 1$:

| Chosen case | Comparison count | Tight growth |
|---|---|---|
| Best | 1 | $\Theta(1)$ |
| Worst | $n$ | $\Theta(n)$ |
| Conditional uniform-position mean | $(n+1)/2$ | $\Theta(n)$ |

Each row can also receive a corresponding Big-O upper bound. Big-O does *not* mean worst case by definition. It is common to report a worst-case upper bound, but that combines two choices rather than making them synonyms.

The average row retains the assumption from Section 1.6: a warning exists and its first position is uniformly distributed. It is not the average over every possible sensor-log model.

*Lecture-slide alignment: Week 10 source PDF, p. 39.*

## 2.6. Worked simplification activity

The source activity asks for the most informative simple classes of four count models. For positive, growing $n$, with all displayed coefficients fixed, the answers are:

| Count model | Dominant term | Tight order |
|---|---|---|
| $8n + 200$ | $n$ | $\Theta(n)$ |
| $3n^2 + 7n + 5$ | $n^2$ | $\Theta(n^2)$ |
| $50 + 4\log_2 n$ | $\log n$ | $\Theta(\log n)$ |
| $n\log_2 n + 10n$ | $n\log n$ | $\Theta(n\log n)$ |

The corresponding Big-O bounds use the same simple functions. In the last row, $n\log_2 n$ eventually dominates $10n$ because their ratio is $(\log_2 n)/10$, which grows. That argument assumes the coefficient 10 remains fixed. A parameter that grows with the input cannot simply be discarded as a constant.

*Lecture-slide alignment: Week 10 source PDF, p. 40.*

## 2.7. Consecutive stages add their work

```python
def two_passes(values):
    """Return the visit count for two consecutive full passes."""
    visits = 0
    for value in values:
        visits += 1  # First pass: n visits.
    for value in values:
        visits += 1  # Second pass: another n visits, not n squared.
    return visits

print(two_passes([2, 4, 6, 8]))  # Four plus four visits.
```

**Code 2.2:** Two passes over the same input are consecutive, not nested.

```
8
```

The first loop visits all $n$ entries and finishes. Then the second loop visits all $n$ entries. The selected total is

$$C(n) = n + n = 2n \in \Theta(n).$$

At $n = 4$, the observed visit count is 8. It is not 16, because the second loop is not inside the first loop. Indentation and execution order, rather than the number of loop keywords on the page, determine the count.

The same reasoning extends to any *fixed* number $k$ of successive full passes: $kn$ visits. If $k$ is itself an independent growing parameter, state $C(n, k) = kn$ instead of discarding it.

*Lecture-slide alignment: Week 10 source PDF, p. 41.*

## 2.8. Independent nested dimensions multiply

```python
def design_pairs(diameters_mm, speeds_rpm):
    """Count candidate pairs; this function does not solve a model."""
    evaluations = 0
    for diameter_mm in diameters_mm:  # n diameter choices.
        for speed_rpm in speeds_rpm:  # m speed choices per diameter.
            evaluations += 1  # Count one considered pair.
    return evaluations

# Source data: 3 diameters and 4 rotational-speed options.
print(design_pairs([6, 8, 10], [500, 1000, 1500, 2000]))
```

**Code 2.3:** Three diameters combined with four speeds give twelve candidates.

```
12
```

For every diameter, the entire speed list is considered. Consequently

$$C(n, m) = \underbrace{n}_{\text{diameters}} \underbrace{m}_{\text{speeds}}.$$

The code counts a considered candidate pair. It does not solve an engineering model, establish feasibility, or determine an optimal design. Those additional tasks can carry their own work.

| How the input grows | Pair count | Growth |
|---|---|---|
| $n$ diameters, $m$ speeds | $nm$ | $\Theta(nm)$ |
| $n$ diameters, four fixed speeds | $4n$ | $\Theta(n)$ |
| $n$ options on both axes | $n^2$ | $\Theta(n^2)$ |

If every candidate requires $N_t$ fixed-cost simulation steps, the simulation work can scale as $nmN_t$. That extra factor cannot be hidden merely because the simulation is packaged behind one function call. The source's phrase "may scale" is important: the exact count depends on the evaluation algorithm and its stopping rule.

*Lecture-slide alignment: Week 10 source PDF, pp. 42–43.*

## 2.9. A fixed inner loop is not a second growing dimension

```python
def repeated_checks(values):
    """Count three fixed actions per element, for exactly 3n actions."""
    checks = 0
    for value in values:
        for repeat in range(3):  # This bound never grows with n.
            checks += 1
    return checks

print(repeated_checks([1, 2, 3, 4]))  # Twelve selected actions.
```

**Code 2.4:** Three checks per value produce exactly three times the list length.

```
12
```

For $n$ input values, the inner body executes three times for each value. The count is $3n$, hence linear. The inner bound is not $n$; its value remains 3 when the dataset grows from ten readings to a million.

This is why the sentence "two nested loops are quadratic" is unreliable. Replace it with an execution argument: *the outer loop visits $n$ values and the inner loop performs three fixed actions for each one.* That is both clearer and more accurate.

*Lecture-slide alignment: Week 10 source PDF, p. 44.*

## 2.10. Dependent bounds require summation

```python
def unique_pair_count(n):
    """Count pairs i < j; assume a nonnegative integer n."""
    pairs = 0
    for i in range(n):
        for j in range(i + 1, n):  # Exclude self and reverse pairs.
            pairs += 1  # Count this unordered pair once.
    return pairs

# Empty and singleton inputs have no distinct-index pairs.
for n in [0, 1, 4, 8]:
    print(n, unique_pair_count(n))
```

**Code 2.5:** Visit each unordered pair of distinct indices once.

```
0 0
1 0
4 6
8 28
```

The inner loop starts at `i + 1`. For $i = 0$ there are $n - 1$ candidates, for $i = 1$ there are $n - 2$, and so on, until the final outer index has no inner candidates. The sum is

$$\begin{aligned} C(n) &= (n-1) + (n-2) + \cdots + 1 + 0 \\ &= \frac{n(n-1)}{2}. \end{aligned}$$

The formula gives zero for $n = 0$ and $n = 1$, as it should: fewer than two positions cannot form a distinct-index pair.

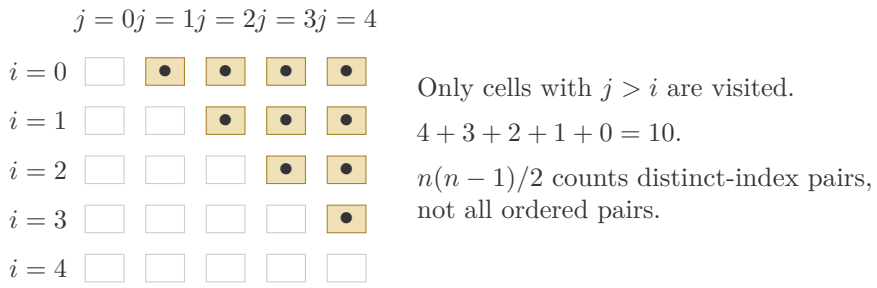



**Figure 2.2.:** The source-page-46 triangular selection for $n = 5$, redrawn as an index grid. Marked cells satisfy $j > i$. The count is $4 + 3 + 2 + 1 + 0 = 10$; diagonal and reversed pairs are excluded.

For large $n$, $n(n-1)/2 = (n^2 - n)/2$ has quadratic order. It is not the same exact count as $n^2$. Excluding duplicate and self comparisons can matter practically even though it does not change the growth class.

The full program still carries loop overhead. For tiny cases with zero selected pair increments, there may be outer-loop iterations and other operations. As before, the formula is the exact *pair* count, not a literal instruction count.

*Lecture-slide alignment: Week 10 source PDF, pp. 45–46.*

## 2.11. A hidden membership scan adds work

```python
def approved_count(installed_ids, approved_ids):
    """Both inputs are lists of identifiers with bounded comparison cost."""
    count = 0
    for sensor_id in installed_ids:  # n installed entries.
        if sensor_id in approved_ids:  # May inspect all m approved IDs.
            count += 1  # Count a match, not every internal comparison.
    return count

print(approved_count([1, 4, 7], [1, 2, 3, 4]))  # IDs 1 and 4 match.
```

**Code 2.6:** One visible loop can contain another scan through list membership.

```
2
```

The output 2 counts installed IDs that are present in the approved list. It does not count the internal membership comparisons. In the worst case, each of $n$ installed IDs requires examination of all $m$ approved entries, producing a bound of $O(nm)$ under bounded-cost identifier comparisons.

For the displayed integers, a left-to-right membership trace requires one comparison for ID 1, four for ID 4, and four for absent ID 7: nine comparisons in that simple trace. This worked trace explains why the observed output 2 must not be misread as a work count. The worst-case bound describes the possible scaling over input cases, not that exact observed trace.

The input types matter. This example uses two *lists*. Changing a list to another data structure changes the assumptions and may require preprocessing and storage, topics reserved for deeper analysis.

*Lecture-slide alignment: Week 10 source PDF, p. 47.*

## 2.12. Conditionals select routes, not both branches

```python
def review_work(n, full_review):
    """Count selected actions; assume n is a nonnegative integer."""
    actions = 0
    if full_review:
        for i in range(n):
            for j in range(n):
                actions += 1  # Full route: n squared actions.
    else:
        for i in range(n):
            actions += 1  # Quick route: n actions.
    return actions  # The two routes do not both run in one call.

print(review_work(4, False), review_work(4, True))
```

**Code 2.7:** The quick and full routes have different selected counts.

```
4 16
```

The quick route executes $n$ actions; the full route executes $n^2$. With either route permitted, the worst-case cost is quadratic for growing $n$. A single call does not execute $n + n^2$ selected actions: only one branch runs. In this example the controlling Boolean is already supplied, so evaluating it does not itself require a scan.

For a more complex condition or helper, any work required to decide the route must also be considered. The general habit is to trace the reachable route and include the operations actually executed, rather than blindly adding every visible block.

*Lecture-slide alignment: Week 10 source PDF, p. 48.*

## 2.13. Constants still matter at practical sizes

The source supplies two exact, normalized cost models:

$$T_A(n) = 100n, \qquad T_B(n) = n^2.$$

| $n$ | $T_A(n)$ | $T_B(n)$ | **Less modelled work** |
|---|---|---|---|
| 10 | 1,000 | 100 | B |
| 100 | 10,000 | 10,000 | Equal |
| 1,000 | 100,000 | 1,000,000 | A |

For positive $n$, equality $100n = n^2$ gives $n = 100$. Below that crossing B has less modelled work; above it A has less. The formulas also coincide at zero, but the meaningful positive-input crossover in the source discussion is 100.

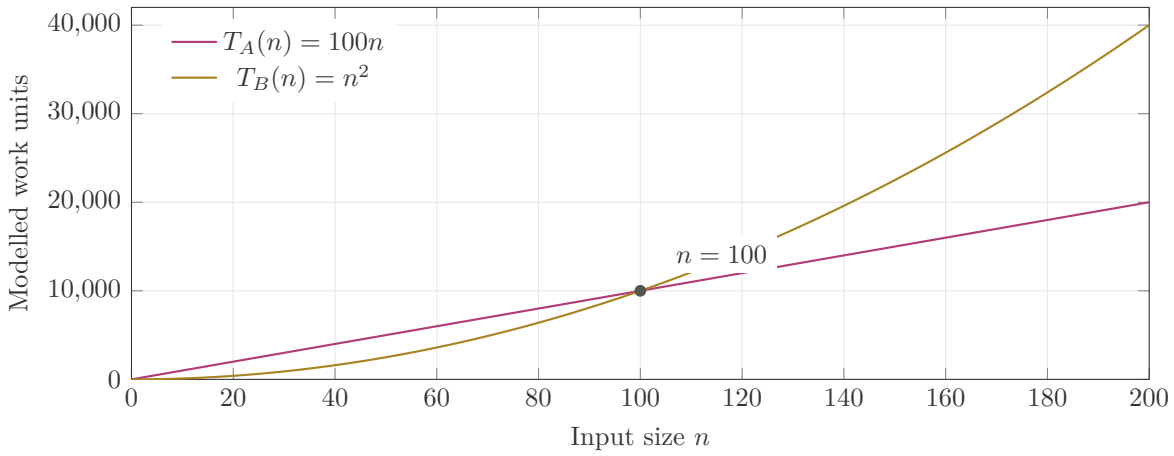



**Figure 2.3.:** Recreated source-page-50 comparison of $100n$ and $n^2$. The axis is modelled work, not measured seconds. The positive-input crossover at 100 comes from these two formulas and is not a universal threshold for linear and quadratic programs.

The linear model eventually grows more slowly, but not at every small input. Big-O intentionally suppresses multiplicative constants; a real implementation choice may require them, as well as memory use, accuracy, and the intended input range. A tiny timing test is not a universal winner selection, and an asymptotic label is not a complete performance report.

*Lecture-slide alignment: Week 10 source PDF, pp. 49–50.*

## 2.14. Logarithm bases and exact counts

By change of base,

$$\log_2 n = \frac{\log_{10} n}{\log_{10} 2}.$$

The denominator is a fixed positive constant. Therefore $O(\log_2 n)$, $O(\log_{10} n)$, and $O(\log n)$ describe the same order. For an *exact* count, the base still matters. Repeated halving suggests base two, while removing decimal digits suggests base ten.

Do not round an exact loop count simply by rounding an arbitrary logarithm to the nearest integer. Integer updates and the stop condition determine whether a floor or ceiling is appropriate. Chapter 3 and the optional exact-count section work through that distinction.

*Lecture-slide alignment: Week 10 source PDF, p. 51.*

## 2.15. Debug the explanation and complete the exit ticket

Four false shortcuts appear in the source debugging clinic. Their repairs are execution questions:

| Shortcut | Correction |
|---|---|
| Two consecutive loops are quadratic. | Add their visit counts; two full passes give $2n$. |
| Two nested loops are always quadratic. | Check whether both bounds grow; a fixed inner bound gives linear work. |
| A built-in call costs one operation. | Inspect the task done inside the call, including scans and copies. |
| Big-O gives exact worst-case time. | State the case, model, and bound separately; Big-O is neither exact seconds nor a case definition. |

**Source exit ticket**

For $C(n) = 5n^2 + 40n + 9$, give the most informative simple Big-O bound and explain why $O(n^3)$ is also valid. Classify eight fixed checks per reading. State the all-pairs count for independent lists of lengths $n$ and $m$.

**Answer and explanation**

The informative simple bound is $O(n^2)$, with tight order $\Theta(n^2)$. For $n \geq 1$, $C(n) \leq 54n^2 \leq 54n^3$, so $O(n^3)$ is valid but loose. Eight fixed checks per reading give $8n$ checks and $\Theta(n)$ growth. Independent all-pairs traversal gives $nm$ pair visits, not automatically $n^2$.

*Lecture-slide alignment: Week 10 source PDF, pp. 52–53.*

# 3 Recognizing Complexity Classes

**Reading route:** Lecture 30; source PDF pages 54–78.

**Learning outcomes**

Recognize constant, logarithmic, linear, and log-linear structures; count quadratic and cubic work using explicit size definitions; distinguish recursive work from recursion depth; recognize exponential branching without executing enormous cases; separate time from retained storage; and verify an improvement that removes repeated work.

*Lecture-slide alignment: Week 10 source PDF, pp. 54–55.*

## 3.1. Attach each class to a reason

A table of names becomes useful only when every name is connected to an execution argument. The source's first map is:

| Bound | Characteristic structure | Size variable |
|---|---|---|
| $O(1)$ | A fixed number of operations, such as one indexed retrieval. | Existing list length. |
| $O(\log n)$ | Repeatedly reduce the remaining size by a fixed factor. | Initial size. |
| $O(n)$ | A fixed amount of work for every item. | Number of readings. |
| $O(n \log n)$ | $n$ items, each receiving logarithmically many stages. | Item count. |
| $O(n^2)$, $O(n^3)$ | Full traversal of two or three growing dimensions. | Points per side. |
| $O(2^n)$ | Visit every combination of $n$ independent binary choices. | Number of choices. |

The table is a selection of important patterns, not a claim that every program belongs to exactly one of these labels. An upper bound can be loose, and a program can depend on multiple independent sizes. The remainder of the chapter supplies the explicit structures behind these introductory examples.

*Lecture-slide alignment: Week 10 source PDF, p. 56.*

## 3.2. Constant work: one known position

```python
def latest_reading(readings):
    """Return the last list element; reject an empty list."""
    if not readings:  # Check emptiness without scanning the list.
        raise ValueError("No readings available")
    return readings[-1]  # One indexed access for an existing list.

# Input creation and full element validation are not inside the function.
print(latest_reading([20.0, 22.0, 25.0]))
```

**Code 3.1:** A constant-work retrieval from an existing list.

```
25.0
```

The function tests whether the list is empty and, for a nonempty input, returns the element at index $-1$. It does not search for the largest or most recent timestamp. "Latest" means the final stored element, so the input's order must already carry the intended meaning.

Under the stated list-access model, the number of operations does not grow with the length of the supplied list. This justifies constant work. The function rejects an empty input with `ValueError`, so both the ordinary and empty routes can be tested.

**Function contract**

The input is already a list. Creating that list and validating every element are outside this function. Adding a scan to validate all values would change the work of the resulting function, even though the final indexed retrieval would still be constant work.

Constant complexity is not zero time. Nor does it promise that this operation has a smaller constant than every operation with a different label at a tiny input size. It states lack of growth with the declared size variable.

*Lecture-slide alignment: Week 10 source PDF, p. 57.*

## 3.3. Logarithmic work: repeatedly halve the size

```python
def halving_steps(size):
    """Count floor-halvings; the input is an integer size >= 1."""
    steps = 0
    while size > 1:
        size //= 2  # Integer division, not subtraction by one.
        steps += 1  # Count body passes, not the final false condition.
    return steps

# For powers of two, the exact count is the base-two exponent.
for size in [1, 8, 16, 32, 1024]:
    print(size, halving_steps(size))
```

**Code 3.2:** Count body passes for the source's integer floor-halving loop.

```
1 0
8 3
16 4
32 5
1024 10
```

The input contract is an integer `size >= 1`. The statement `size //= 2` assigns the floor of half the previous integer. The loop does not subtract one repeatedly; it discards approximately half of what remains on every pass.

For a power of two $n = 2^k$, the successive sizes are $2^k, 2^{k-1}, \ldots, 2, 1$. Exactly $k$ body passes occur, giving

$$\frac{n}{2^k} = 1 \quad \Longrightarrow \quad k = \log_2 n.$$




**Figure 3.1.:** The source-page-59 trace: five arrows take size 32 to 1. The six displayed states must not be confused with the five updates between them.

For an arbitrary positive integer, this exact floor-halving loop performs $\lfloor \log_2 n \rfloor$ body passes. For example, 10 becomes 5, then 2, then 1: three passes. Doubling the initial positive integer adds one floor-halving pass. At size 1 there is no body execution, but the condition is still tested.

A loop's keyword does not determine its complexity. A `while` loop that subtracts one can have a linear number of updates; one that divides by two can have logarithmic updates. The progress rule supplies the argument.

*Lecture-slide alignment: Week 10 source PDF, pp. 58–59.*

## 3.4. Interval refinement is another halving process

In Week 3, bisection repeatedly reduced a numerical bracket. If the starting width is $L_0$, then after $k$ exact halvings the width is

$$L_k = \frac{L_0}{2^k}.$$

To require $L_k \le \varepsilon_x$, solve

$$2^k \ge \frac{L_0}{\varepsilon_x}, \qquad k \ge \left\lceil \log_2\left(\frac{L_0}{\varepsilon_x}\right) \right\rceil .$$

When the starting width already meets the requirement, zero halvings are sufficient; the minimum nonnegative count is therefore the larger of zero and that ceiling.

For the source's example, $L_0 = 8\,\text{s}$ and $\varepsilon_x = 0.125\,\text{s}$. Their ratio is 64, so six halvings suffice:

$$8,\ 4,\ 2,\ 1,\ 0.5,\ 0.25,\ 0.125 \text{ s}.$$

The target is a final *interval width*, not a residual threshold and not the number of digits to print. A returned midpoint of a genuine bracket has a different error statement: its distance to a root in that bracket is bounded by half the width. Do not silently interchange these requirements.

**Engineering check**

The halving count assumes each model evaluation has bounded cost and the bracket can keep shrinking. A residual-only stopping rule, an expensive model call, or finite-precision stagnation requires separate analysis. The source explicitly separates these limitations from the width count.

*Lecture-slide alignment: Week 10 source PDF, p. 60.*

## 3.5. Linear work: integrate a sampled power record

The energy example uses a piecewise-constant model: each supplied power value is applied over one full interval of length $\Delta t$. Then

$$E \approx \sum_{i=1}^{n} P_i \Delta t.$$

Power in watts multiplied by time in seconds produces energy in joules. The same unit convention is reflected in the variable names.

```python
def total_energy_j(power_w, dt_s):
    """Return sum(P_i * dt); each power value lasts dt_s seconds."""
    energy_j = 0.0  # One running energy total, in joules.
    for value_w in power_w:
        energy_j += value_w * dt_s  # W times s is J; one update/sample.
    return energy_j

# Source model: three full two-second intervals, not just endpoints.
print(total_energy_j([10.0, 20.0, 30.0], 2.0))
```

**Code 3.3:** One multiplication and one accumulation per supplied interval.

```
120.0
```

The three contributions are 20 J, 40 J, and 60 J, giving 120 J. Here three power entries represent three two-second intervals. They are not merely three time-point endpoints from which two intervals should be inferred.

The work is linear in $n$ under fixed-cost arithmetic: one multiplication and one energy accumulation per input value. A selected count of *multiplications* is $n$; a selected count of *multiplications plus accumulations* is $2n$. Both describe linear growth, but they are not the same exact selected count.

The function retains a fixed number of running scalars and does not store every intermediate contribution. This is a computational statement about the supplied model, not a claim about the measurement accuracy or whether a piecewise-constant model is adequate for a real power signal.

*Lecture-slide alignment: Week 10 source PDF, p. 61.*

## 3.6. Log-linear work: reset the stages for each item

```python
def staged_work(n):
    """Counting model, not sorting; assume integer n >= 1."""
    work = 0
    for item in range(n):
        size = n  # Reset here so every item receives all stages.
        while size > 1:
            size //= 2  # One fixed-factor reduction.
            work += 1  # n * floor(log2(n)) selected increments.
    return work

for n in [8, 16, 32]:  # Small, hand-checkable examples.
    print(n, staged_work(n))
```

**Code 3.4:** A counting model with logarithmically many stages for each item.

```
8 24
16 64
32 160
```

The outer loop visits $n$ items. Inside each pass, `size = n` resets the halving process. Therefore every item receives the same $\lfloor \log_2 n \rfloor$ counted updates:

$$C(n) = n\lfloor \log_2 n \rfloor.$$

For $n = 8$, the count is $8 \times 3 = 24$; for 16 it is 64; for 32 it is 160. This gives tight order $\Theta(n \log n)$ for growing $n$.

The position of the reset is essential. Moving `size = n` outside the outer loop would cause only the first item to perform the full halving process. Later items would see the already-reduced size. That would be a different execution structure, not a harmless rearrangement of the same algorithm.

At $n = 1$, the selected inner increment count is zero, but the outer loop, initialization, and tests still execute. The expression describes selected work, not zero runtime at that input.

**Technical clarification**

This program is **not a sorting algorithm**. It is an intentionally simple counting model that makes the $n \log n$ pattern visible. Searching and sorting implementations belong to the dedicated Week 12 unit.

*Lecture-slide alignment: Week 10 source PDF, p. 62.*

## 3.7. Quadratic work: compare every ordered pair

```python
def maximum_gap(values):
    """Return (largest absolute gap, pair checks); nonempty finite list."""
    largest_gap = 0.0
    checks = 0
    for a in values:
        for b in values:  # Include self-pairs and reversed pairs.
            checks += 1  # Exactly n*n pair evaluations.
            largest_gap = max(largest_gap, abs(a - b))
    return largest_gap, checks

print(maximum_gap([20.0, 23.0, 25.0]))  # Gap 5.0; nine pair checks.
```

**Code 3.5:** All pairs, including self-pairs and reversed pairs, are evaluated.

```
(5.0, 9)
```

For each of $n$ choices of `a`, the loop visits all $n$ choices of `b`. There are $n^2$ pair checks. In the source data, the maximum absolute difference is 5.0, obtained by comparing 20.0 with 25.0, and the selected work count is 9.

This traversal includes $(a, a)$ and both $(a, b)$ and $(b, a)$. The unique-pair function in Section 2.10 deliberately did not. Both have quadratic order but different exact counts and different pair definitions.

**Function contract**

The stated input is a nonempty finite numerical list. The source function assumes that precondition; it does not explicitly reject an empty list. Its mechanical result for an out-of-contract input is not evidence that the intended maximum-gap question has been answered meaningfully.

Doubling $n$ multiplies the pair count by four. It does not follow that every real elapsed-time measurement will increase by exactly four, because that would require all other costs and measurement effects to scale proportionally too.

*Lecture-slide alignment: Week 10 source PDF, p. 63.*

## 3.8. Cubic work and the choice of dimension

```python
def grid_locations(n):
    """Count visits in an n by n by n grid; integer n >= 0."""
    visits = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                visits += 1  # Only count; no grid array is allocated.
    return visits

print(grid_locations(4))  # 4**3 = 64 points.
print(grid_locations(8))  # 8**3 = 512, an eightfold increase.
```

**Code 3.6:** Count grid visits without allocating a grid.

```
64
512
```

For $n$ points in each of three independent directions, the number of locations is $n^3$. The code counts one visit to each location. It does not allocate a three-dimensional array and does not evaluate a physical field there.

The source comparison doubles points per side from 4 to 8, increasing locations from 64 to 512: a factor of eight. If the size variable is instead the total number of points $N = n^3$, then visiting each location once is linear in $N$. Always name which quantity is growing.

| Change in $n$ | One pass | Square grid | Cube grid |
|---|---|---|---|
| Double $n$ | $\times 2$ | $\times 4$ | $\times 8$ |
| Multiply $n$ by ten | $\times 10$ | $\times 100$ | $\times 1000$ |

**Polynomial growth** uses a fixed exponent, as in $n^p$ for a fixed $p$. A polynomial label does not automatically make a computation practical: the exponent may be large, each evaluation may be costly, or the required input may exceed available resources.

*Lecture-slide alignment: Week 10 source PDF, pp. 64–65.*

## 3.9. Exponential branching: count leaves and calls

```python
def visit_binary_choices(n):
    """Return (leaves, calls); use only small nonnegative integer n."""
    if n == 0:
        return 1, 1  # One completed configuration and one base call.
    left, left_calls = visit_binary_choices(n - 1)
    right, right_calls = visit_binary_choices(n - 1)
    # Include both subtrees and this call, not just the leaves.
    return left + right, 1 + left_calls + right_calls

print(visit_binary_choices(4))  # 16 leaves, 31 total calls.
```

**Code 3.7:** This function visits both subtrees rather than just computing their size.

```
(16, 31)
```

There are two smaller calls at every nonterminal node. If there are $n$ binary choices, then after all choices have been made there are $2^n$ leaf configurations. Every leaf is visited by this recursive program.

The total number of calls includes the internal decisions as well:

$$1 + 2 + 4 + \cdots + 2^n = 2^{n+1} - 1.$$

For $n = 4$, that gives 16 leaves and 31 total calls, matching the output. A selected count of leaf visits is not the same as a count of all calls, although both have exponential order in this example.

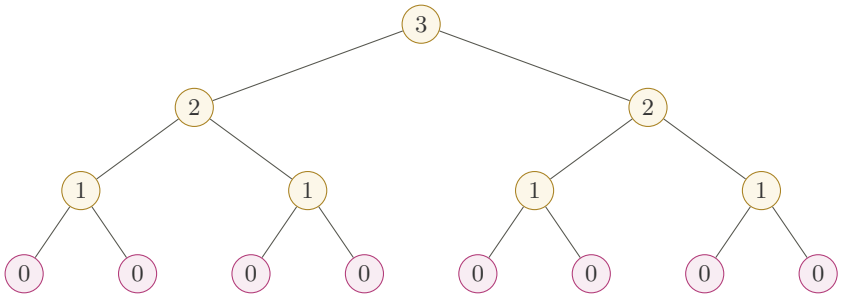



**Figure 3.2.:** Added explanatory trace of the same binary-choice rule for $n = 3$: 8 leaves and 15 calls. The smaller tree is drawn to expose the levels; the source's executable demonstration uses $n = 4$.

Merely evaluating `2**n` calculates a number. It does not enumerate every configuration. The amount of a returned number is not an operation count unless the program actually performs the corresponding work.

Do not try huge inputs to confirm the growth. Trace or run a few small cases and derive the general count. Exhaustively visiting an exponentially growing tree can become impractical long before the input integer appears large.

*Lecture-slide alignment: Week 10 source PDF, p. 66.*

## 3.10. Fibonacci exposes repeated subproblems

Retain the convention introduced earlier in the course:

$$F(0) = F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

The counted implementation returns both the sequence value and the number of executed calls.

```python
def fibonacci_counted(n):
    """Return (F(n), calls); assume n >= 0 and F(0) = F(1) = 1."""
    if n == 0 or n == 1:
        return 1, 1  # Each base case is one executed call.
    a, ca = fibonacci_counted(n - 1)  # First smaller subproblem.
    b, cb = fibonacci_counted(n - 2)  # Overlaps earlier subproblems.
    return a + b, 1 + ca + cb  # Add the current call to both counts.

for n in [5, 10, 20]:  # Do not use very large n in this naive method.
    print(n, fibonacci_counted(n))
```

**Code 3.8:** The source convention and the total-call count are both explicit.

```
5 (8, 15)
10 (89, 177)
20 (10946, 21891)
```

For example, evaluating $F(4)$ needs $F(3)$ and $F(2)$, while evaluating $F(3)$ already needs $F(2)$. The naive method calculates these overlapping subproblems repeatedly instead of retaining a previously computed answer.

Let $A(n)$ count all calls. Then

$$A(0) = A(1) = 1, \qquad A(n) = 1 + A(n-1) + A(n-2).$$

The initial 1 counts the present call; omitting it would count only descendants. This recurrence is a worked explanation of the counters, not a new sequence convention.

The source states $O(2^n)$ as a convenient upper bound and explicitly says it is not tight for this naive Fibonacci algorithm. Do not turn that statement into $\Theta(2^n)$. It is enough at this stage to recognize rapidly growing repeated work and the difference between a useful upper bound and an exact or tight count.

*Lecture-slide alignment: Week 10 source PDF, p. 67.*

### 3.10.1. An iterative alternative

```python
def fibonacci_iterative(n):
    """Return (F(n), sequence additions); integer n >= 0; F(0)=F(1)=1."""
    previous, current = 1, 1  # The two starting sequence values.
    additions = 0
    for index in range(2, n + 1):
        previous, current = current, previous + current
        additions += 1  # Count sequence updates, not recursive calls.
    return current, additions

for n in [5, 10, 20]:  # Same sequence indices as the recursive example.
    print(n, fibonacci_iterative(n))
```

**Code 3.9:** Retain two recent values instead of re-solving smaller cases.

```
5 (8, 4)
10 (89, 9)
20 (10946, 19)
```

The right-hand side of the multiple assignment is evaluated using the old values. Then the old `current` becomes `previous`, and the sum becomes the new `current`. Thus only two sequence values need to be retained.

For integer $n \geq 2$, the loop performs $n - 1$ sequence additions. For $n = 0$ or 1, it performs none and returns the required starting value 1. The source comparison is:

| $n$ | $F(n)$ | **Recursive calls** | **Iterative additions** |
|---|---|---|---|
| 5 | 8 | 15 | 4 |
| 10 | 89 | 177 | 9 |
| 20 | 10,946 | 21,891 | 19 |

Calls and sequence additions are *different selected operations*. Dividing the two columns is not a measured speedup. The structural comparison shows why the iterative method avoids repeated work. Under bounded-cost arithmetic it is linear in $n$, retaining a fixed number of sequence-value slots; the growing bit lengths of the integers remain outside that simplified storage model.

*Lecture-slide alignment: Week 10 source PDF, p. 68.*

## 3.11. Recursion is not itself a complexity class

```python
def factorial(n):
    """Return n!; assume a small nonnegative integer n."""
    if n == 0:
        return 1  # The terminal call does not multiply again.
    return n * factorial(n - 1)  # One child call and one multiplication.

print(factorial(5))  # Five multiplications; six calls including n=0.
# The magnitude of n! is not the number of executed multiplications.
```

**Code 3.10:** One smaller call per level gives a linear call chain.

```
120
```

For input $n$, the calls have arguments $n, n-1, \ldots, 1, 0$. There are $n+1$ calls and $n$ multiplications. At the deepest point, all $n+1$ frames are active because their pending products cannot finish until the base case returns.

This is linear work and linear frame count under the declared scalar-state, unit-cost arithmetic model. It is not exponential simply because it uses recursion. It is also not factorial work simply because the returned value is $n!$.

**Common mistake**

Do not infer runtime from the name of a function or the magnitude of its answer. Follow how many calls actually execute, what work each performs, and what must stay live while later calls are active.

The actual integer result grows in bit length, and its arithmetic and storage can require additional resources. The source flags that limitation rather than claiming that all-size arbitrary-precision arithmetic has constant cost.

*Lecture-slide alignment: Week 10 source PDF, p. 69.*

## 3.12. Compare normalized growth models

The source evaluates mathematical models with leading coefficient one:

| **Model** | $n = 10$ | $n = 20$ | $n = 40$ |
|---|---|---|---|
| $\log_2 n$ | 3.32 | 4.32 | 5.32 |
| $n$ | 10 | 20 | 40 |
| $n \log_2 n$ | 33.2 | 86.4 | 212.9 |
| $n^2$ | 100 | 400 | 1,600 |
| $n^3$ | 1,000 | 8,000 | 64,000 |
| $2^n$ | 1,024 | 1,048,576 | $1.10 \times 10^{12}$ |

The logarithmic and exponential rows behave very differently even though both contain the number two. Doubling $n$ adds one to $\log_2 n$, but changes $2^n$ to $2^{2n}$.

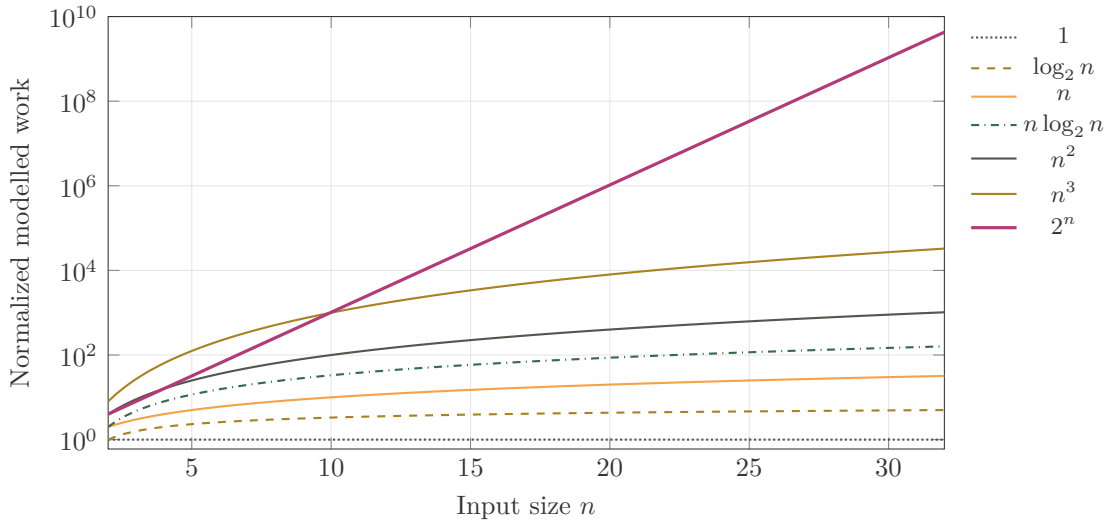



**Figure 3.3.:** Recreated source-page-71 normalized models for $2 \leq n \leq 32$. The vertical axis is logarithmic so several orders remain visible beside the exponential curve. These curves are not timing data or a prediction of seconds.

The plotted curves and tabulated numbers do not include hardware-dependent coefficients, loop overhead, or costs of model evaluation. They explain growth separation. A specific implementation still needs its own contract, count, and measurements.

*Lecture-slide alignment: Week 10 source PDF, pp. 70–71.*

## 3.13. Worked activity: a normalized work budget

The source's hypothetical budget permits at most $10^6$ normalized work units and assumes the *exact* model $C(n) = n^2$. Solving the inequality gives

$$n^2 \leq 10^6 \quad \Longrightarrow \quad n \leq 1000,$$

so the largest permitted nonnegative integer size is 1000. Doubling it requires four times the work budget.

```
# Added arithmetic check of the exact model C(n) = n**2.
# This is NOT a time measurement or a generic consequence of Big-O.
budget = 1_000_000
largest_n = 1_000  # Derived by solving n**2 <= budget.
assert largest_n**2 <= budget
assert (largest_n + 1)**2 > budget
print("Largest permitted n:", largest_n)
print("Budget multiplier for double n:", (2 * largest_n)**2 / budget)
```

**Code 3.11:** Added arithmetic verification of the exact model boundary.

```
Largest permitted n: 1000
Budget multiplier for double n: 4.0
```

Knowing only $C(n) \in O(n^2)$ would not establish that numerical limit. The multiplicative constant, lower-order work, and starting threshold are not specified by the upper-bound label.

A model such as $100n^2$ is still quadratic but exhausts the same normalized budget at a much smaller size.

*Lecture-slide alignment: Week 10 source PDF, p. 72.*

## 3.14. Separate time, output, and additional state

| Task | Time model | Extra retained storage |
|---|---|---|
| Return the final running total | $O(n)$ | $O(1)$ scalar slots. |
| Return every cumulative total | $O(n)$ | $O(n)$ requested output values. |
| Recursive factorial | $O(n)$ | $O(n)$ active call frames. |

The table follows the source convention: supplied input storage is excluded, but retained output is included. "Auxiliary storage excluding output" would be a different convention, so it must be named when used. A complete list of cumulative totals cannot be stored in a fixed number of scalar slots when the required list has $n$ entries.

The factorial row uses the unit-cost and scalar-state model. Linear frame count does not automatically equal a fixed number of bytes per frame when the stored integers themselves grow. Keep the modelling assumption attached to the claim.

*Lecture-slide alignment: Week 10 source PDF, p. 73.*

## 3.15. Integrated improvement: all cumulative totals

Suppose the input is `[2.0, 3.0, 4.0]`. The required output is the total through each sample, in original order:

$$[2.0,\ 2.0+3.0,\ 2.0+3.0+4.0] = [2.0, 5.0, 9.0].$$

Returning just 9.0 would not satisfy the same task. The output must also preserve the input rather than modifying it.

### 3.15.1. Repeated slicing and summation

```python
def prefix_totals_repeated(values):
    """Return all prefix totals in order without mutating the input."""
    totals = []
    for end in range(1, len(values) + 1):
        # Copy and rescan prefixes of lengths 1, 2, ..., n.
        totals.append(sum(values[:end]))
    return totals

print(prefix_totals_repeated([2.0, 3.0, 4.0]))
```

**Code 3.12:** Correct prefix outputs obtained by repeatedly copying and rescanning.

```
[2.0, 5.0, 9.0]
```

At successive values of `end`, the slices contain 1, 2, through $n$ entries. The summations therefore visit

$$C_{\text{repeated}}(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

elements in total. The slices also copy the corresponding numbers of references. The expression is the selected count for the summation visits, not an instruction count that already includes copying and every other action.

The outer loop is visible, but the repeated work inside `sum(values[:end])` is easy to overlook. The function is quadratic under the stated bounded-operation model even though its body is short.

*Lecture-slide alignment: Week 10 source PDF, p. 74.*

### 3.15.2. Reuse the previous subtotal

```python
def prefix_totals_running(values):
    """Return all prefix totals in one pass without changing the input."""
    totals = []  # The n requested outputs still need to be stored.
    running = 0.0
    for value in values:
        running += value  # Extend rather than recompute the subtotal.
        totals.append(running)  # Store the total through this sample.
    return totals

print(prefix_totals_running([2.0, 3.0, 4.0]))
```

**Code 3.13:** One traversal retains the subtotal already computed.

```
[2.0, 5.0, 9.0]
```

The running method extends one subtotal by the next value and stores the result. It visits the $n$ input values once. For the small example, the trace is:

| New value | Running subtotal | Output so far |
|---|---|---|
| 2.0 | 2.0 | `[2.0]` |
| 3.0 | 5.0 | `[2.0, 5.0]` |
| 4.0 | 9.0 | `[2.0, 5.0, 9.0]` |

The count becomes $C_{\text{running}}(n) = n$. Both methods still retain $n$ output values. Improving time did not remove the requested output-storage requirement.

**Numerical agreement still needs a criterion**

The displayed data give exactly matching results. This does not establish bit-for-bit equality of every floating-point prefix on every Python implementation. The recovered slide speaker notes flag possible differences between summation implementations. Preserve the original accuracy requirement and use independent expected answers or a justified tolerance for non-exact cases; do not replace numerical testing with a complexity argument.

*Lecture-slide alignment: Week 10 source PDF, p. 75.*

### 3.15.3. Check the contract before trusting the speed

```python
# Added tests: the two implementations are the ones printed in the notes.
from reading_models import prefix_totals_repeated, prefix_totals_running

# Expected outputs are written independently, not obtained from a method.
cases = [([], []), ([4.0], [4.0]),
         ([2.0, 3.0, 4.0], [2.0, 5.0, 9.0]),
         ([2.0, -3.0, 4.0], [2.0, -1.0, 3.0])]
for method in [prefix_totals_repeated, prefix_totals_running]:
    for values, expected in cases:
        before = values.copy()  # Record the non-mutation expectation.
        actual = method(values)
        assert actual == expected  # Exact for these small test values.
        assert len(actual) == len(values)  # One total per input sample.
        assert values == before  # Original data are unchanged.
print("Both implementations passed four known-answer cases.")
```

**Code 3.14:** Added known-answer and non-mutation checks for both source implementations.

```
Both implementations passed four known-answer cases.
```

The test driver uses the matching functions in `reading_models.py`. It includes empty input, a singleton, an ordinary positive record, and signed data. The expected lists are explicitly stated, rather than generated by one candidate method and then treated as truth for the other. Length and input preservation are checked independently of the values.

Only after tests pass should we compare performance at representative sizes. Retain the required output, order, units, and accuracy; identify repeated scans and copies; change the implementation deliberately; rerun tests; measure; and record the remaining time–storage trade-off. These steps carry Week 7's testing discipline into efficiency work.

*Lecture-slide alignment: Week 10 source PDF, pp. 75–76.*

## 3.16. Correct the quick-poll misconceptions

Every statement in the source quick poll needs correction as a universal rule.

| Claim | Counterexample or repair |
|---|---|
| Every `while` loop is linear. | Repeated floor-halving gives logarithmic body passes. |
| Every recursive function is exponential. | The factorial call chain uses one smaller call per level. |
| Same Big-O means same runtime. | Fixed multipliers and implementation details can differ. |
| A built-in call always costs constant work. | `sum()` scans the numerical input. |
| A fast result is acceptable even if tests fail. | Speed cannot substitute for the required output or accuracy. |

A corrected explanation must name the algorithmic structure, not just replace one memorized label with another. Being able to supply a concrete counterexample is a strong check on understanding.

*Lecture-slide alignment: Week 10 source PDF, p. 77.*

## 3.17. Lecture 30 exit ticket and synthesis

| Source pattern | Class and reason |
|---|---|
| One indexed lookup | $O(1)$: fixed work for an existing list. |
| $n$ independent fixed-cost checks | $O(n)$: one fixed action per item. |
| Halve size until it reaches 1 | $O(\log n)$: fixed-factor reduction. |
| Visit all $n \times n$ pairs | $O(n^2)$: two growing dimensions. |
| Visit all binary-choice leaves | $O(2^n)$: one leaf per binary configuration. |

**A small source typesetting issue**

The fourth row on source page 78 displays a malformed multiplication expression ("nimesn"). In the table above it is rendered as $n \times n$, matching the all-pairs example and the recovered speaker-note answer. This is a labelled notation repair, not a changed algorithm.

The prefix-total improvement must be tested because fewer visits do not prove that outputs, order, empty-input behavior, data preservation, or numerical accuracy remain correct. A clear count explains the opportunity; tests and appropriate measurements support the revised implementation.

**Section summary**

Recognize a class by following execution: fixed work, one pass, repeated halving, multiple growing dimensions, or branching calls. State what the size means, the input case, and the work model. Keep output magnitude, active memory, total calls, and elapsed seconds as separate quantities. Optimize repeated work only while preserving the engineering task.

*Lecture-slide alignment: Week 10 source PDF, pp. 78,87.*

# 4 Optional Reference and Preparation

**Reading route:** Optional and reference slides, source PDF pages 79–86. These sections clarify the existing examples; they do not add assessment requirements.

## 4.1. Exact counts include the final condition test

Suppose `while size > 1` executes its body $k$ times and stops normally because the condition becomes false. The condition is evaluated $k + 1$ times: once before each body pass and once to detect termination. The final false test is not a body execution.

| Floor-halving input | Body passes | Condition tests |
|---|---|---|
| $n = 8$ | 3 | 4 |
| $n = 10$ | 3 | 4 |
| $n = 1$ | 0 | 1 |

For the exact update `size //= 2` on integer $n \geq 1$, the body count is $\lfloor \log_2 n \rfloor$. To see why, let $k = \lfloor \log_2 n \rfloor$, so $2^k \leq n < 2^{k+1}$. After $k$ floor-halvings the size is 1; before the last needed update it was still larger than 1. This yields a floor, unlike the ceiling used to guarantee that a continuously halved interval meets a specified width.

```
# Added tracing driver for the unchanged floor-halving rule.
for initial in [1, 10, 64]:
    size = initial
    history = [size]  # This diagnostic list is extra, non-core storage.
    updates = 0
    while size > 1:
        size //= 2
        updates += 1
        history.append(size)  # Retain each state only to explain it.
    # The original while condition is tested once more than its body.
    print(history, "updates:", updates, "tests:", updates + 1)
```

**Code 4.1:** Added trace driver retaining the source update and stopping condition.

```
[1] updates: 0 tests: 1
[10, 5, 2, 1] updates: 3 tests: 4
[64, 32, 16, 8, 4, 2, 1] updates: 6 tests: 7
```

The driver stores a history to make states visible. That history is *additional diagnostic storage*; it is not part of the original scalar-only `halving_steps()` function. The count $k + 1$ assumes normal false-condition termination. A loop ending by `break`, exception, or `return` requires its own route trace.

*Lecture-slide alignment: Week 10 source PDF, p. 79.*

## 4.2. A standard helper for microbenchmarks

```python
# Optional standard-library helper: measure batches of the same task.
from timeit import repeat

values = [1.0] * 100_000  # The input is already built when timing starts.

def task():
    # A no-argument callable lets repeat invoke the same calculation.
    return sum(values)

assert task() == 100_000.0  # Validate before any recorded timing.
trials_s = repeat(task, repeat=5, number=20)
# A trial contains 20 calls; convert its duration to seconds per call.
print("Best observed seconds per call:", min(trials_s) / 20)
```

**Code 4.2:** The two repetition parameters answer different questions.

Here `repeat=5` asks for five recorded trials, and `number=20` means 20 task calls inside each trial. The function `repeat()` returns trial durations, not individual-call durations. Dividing the chosen trial time by 20 reports an average duration per call within that particular trial. Taking the minimum first produces the best observed batch average, not a deadline guarantee.

The zero-argument `task()` function uses the already-created list. Its assertion runs before recording. This example measures the wrapper call and the summation, with input construction excluded. It is a *microbenchmark*: a deliberately narrow measurement, not a complete end-to-end workflow.

The source names the official `time` and `timeit` documentation for further details. This reading note retains the displayed protocol rather than silently changing how trials are recorded or which summary is reported. The script's output is variable and is not supplied as a fixed numerical answer.

*Lecture-slide alignment: Week 10 source PDF, p. 80.*

## 4.3. Integer magnitude and representation length

The source's optional size discussion distinguishes the magnitude of a positive integer $n$ from the number $b$ of bits used to represent it. More precisely,

$$b = \lfloor \log_2 n \rfloor + 1, \qquad 2^{b-1} \leq n < 2^b.$$

This exact expression expands the source's approximate phrase "about $\log_2 n$ bits." A loop that counts down from $n$ by ones makes $n$ updates in the unit-cost arithmetic model. Expressed in terms of representation length, that update count can grow exponentially in $b$.

For example, storing the positive integer 1024 needs 11 bits in its binary magnitude representation, but counting down from it by one requires 1024 updates. The number of written digits and the number of requested repeated actions are different quantities.

Large Python integers also require increasing arithmetic and storage resources. The Week 10 examples intentionally use the simpler bounded-number model to expose the main structures. That model must not be mistaken for an all-size bit-complexity analysis of factorial or Fibonacci.

*Lecture-slide alignment: Week 10 source PDF, p. 81.*

## 4.4. Reference costs for familiar operations

| Operation | Source cost model | Qualification |
|---|---|---|
| `len(a)`, `a[i]` | $O(1)$ | An existing Python list; a valid index. |
| `a.copy()`, `sum(a)` | $O(n)$ | Copy references or process numerical values. |
| `x in a` | $O(n)$ worst case | List membership, not every container type. |
| `a.append(x)` | $O(1)$ amortized | One individual resizing operation can cost more. |
| `mapping[key]` | $O(1)$ average case | Not an unconditional worst-case guarantee. |

**Amortized** and **average case** do not mean the same thing. In the source's append qualification, the point is to account for occasional expensive resizes across a sequence of append operations. An average-case lookup statement carries assumptions about the lookup model rather than promising constant work for every possible case.

The table assumes bounded-cost element operations. Long identifiers, large integers, or custom comparison behavior can add work. Use this table as a reference for the declared classroom model, not as a rule that every line containing one of these operations takes identical real time.

*Lecture-slide alignment: Week 10 source PDF, p. 82.*

## 4.5. An optional factorial-growth contrast

The reference slide adds $O(n!)$ for visiting every ordering of $n$ distinct items, with bounded counted work per visit. There are $n$ choices for the first position, $n-1$ for the second, and so on, giving

$$n(n-1)\cdots 2\cdot 1 = n!$$

orderings. This is a visit-count description. Copying a complete ordering or performing a size-dependent engineering evaluation on each one would add further work.

This is deliberately different from the recursive `factorial(n)` program. That program calculates one numerical value using a chain of $n$ multiplications; it does not visit all $n!$ orderings. The source includes the factorial-growth label as an optional contrast, not as a requirement to implement a permutation generator.

| Growth family | Structural reason to remember |
|---|---|
| Constant | Fixed number of actions. |
| Logarithmic | Fixed-factor reduction. |
| Linear | Fixed work per item. |
| Log-linear | Full-size work across logarithmically many stages. |
| Quadratic / cubic | Two or three growing dimensions. |
| Exponential | Every binary-choice configuration is visited. |
| Factorial | Every ordering of distinct items is visited. |

*Lecture-slide alignment: Week 10 source PDF, p. 83.*

## 4.6. Worked preparation for Week 11

The five source preparation tasks are practice, not a new graded assignment.

### 4.6.1. Three consecutive scans

The exact selected visit count is $3n$, with linear growth. The three stages add; none is performed inside every iteration of another.

```python
# Added solution: count the visit chosen in the practice question.
def three_scans(values):
    """Return 3n element visits for three successive passes."""
    visits = 0
    for value in values:
        visits += 1  # First complete pass.
    for value in values:
        visits += 1  # Second complete pass.
    for value in values:
        visits += 1  # Third complete pass.
    return visits

print(three_scans([2, 4, 6, 8]))  # Twelve visits.
```

**Code 4.3:** Added execution check for the source practice question.

```
12
```

### 4.6.2. Fixed inner work versus triangular work

A fixed inner count $r$ gives $rn$ selected actions, linear when $r$ stays fixed. The unique-pair triangular loop gives $n(n-1)/2$, quadratic for growing $n$. A half factor changes the exact count but not the dominant order.

### 4.6.3. Trace the logarithmic counter

For inputs 1, 10, and 64, the original floor-halving loop performs 0, 3, and 6 updates respectively. The trace in Section 4.1 displays all states and distinguishes updates from condition tests.

### 4.6.4. Test cumulative totals

The empty record should produce `[]`. The signed record `[2.0, -3.0, 4.0]` should produce `[2.0, -1.0, 3.0]`. Check both the values and the unchanged original record; the driver in Section 3.15 does so.

### 4.6.5. Write an efficiency report

A possible four-sentence report is:

> The input contains $n$ finite numerical readings. The running-prefix implementation adds each reading to a subtotal once and appends one result per reading. Every valid input of length $n$ therefore receives $n$ numerical updates under the bounded-cost

> model, with linear work and $n$ retained outputs. Known-answer, empty, singleton, signed-input, and non-mutation tests support the implementation; timing is reported separately for any measured environment.

The final sentence distinguishes verified behavior from an unperformed timing claim. Do not invent an elapsed time to make a report appear complete.

*Lecture-slide alignment: Week 10 source PDF, p. 84.*

## 4.7. Where the course goes next

The source identifies Week 11 as the continuation on deeper complexity comparisons, hidden work, and practical time–space trade-offs. Week 12 develops search and sort implementations, their requirements, and their costs. These notes do not replace that later sequence with a new methods unit.

The closing message is that a correct answer should also be practical: understand the work before optimizing; state the input size; explain growth; measure the implementation; and keep the engineering result correct.

*Lecture-slide alignment: Week 10 source PDF, pp. 85–87.*

# 5 Cumulative Practice with Worked Solutions

The following twelve questions are **additional reading-note practice** based on the source examples. They are not examination announcements or new graded requirements. Attempt each question before reading its answer. A correct symbol without an execution argument is not a complete solution.

## 5.1. P1: output events versus executed comparisons

A five-reading log is `[20.0, 30.0, 28.0, 30.0, 25.0]`. Apply `count_warnings()` with a limit of 30.0. Predict both returned values and explain why one can be zero while the other is not.

**Answer and explanation**

The returned pair is `(0, 5)`. No value is strictly above the limit, but all five comparisons are needed to establish that fact. Equality at 30.0 does not warn. A counter incremented only inside the warning branch would not measure the full comparison work.

## 5.2. P2: preserve an early-exit contract

For `[190.0, 200.0, 205.0, 300.0]`, locate the first pressure strictly above 200 kPa. Give the value, zero-based index, and number of comparisons. Could this early-exit calculation also report the total number of warnings?

**Answer and explanation**

The value is 205.0 kPa, its index is 2, and three comparisons execute. The value at 200.0 is permitted by the strict rule. Early exit has not examined the final 300.0 reading, so it cannot establish the complete warning total. The correct total is 2 only after the full data are inspected.

## 5.3. P3: justify the average-case assumption

A first-warning search uses 1 through $n$ comparisons when a warning exists. A student writes $(n+1)/2$ as the average for every possible collection of logs. What is missing? Under the source assumption, what is the average for $n = 12$?

**Answer and explanation**

The probability model is missing. The source assumes that a warning exists and its first position is uniformly distributed over the $n$ positions. Under that model, the average is $(12+1)/2 = 6.5$ comparisons. It is an expectation over possible runs, not a claim that a run executes half a comparison. Other position distributions and absent-warning cases change the expectation.

## 5.4. P4: compute the selected three-stage count

Evaluate `selected_additions()` at $n = 0$ and $n = 4$ without executing it. Which stage controls growth for large $n$? What does the return value fail to count?

**Answer and explanation**

The count is $1000 + n + 2n^2$. It gives 1000 at $n = 0$ and $1000 + 4 + 32 = 1036$ at $n = 4$. The quadratic stage dominates eventually. The return value counts only the selected increment statements, excluding function entry, initialization, loop machinery, and returning. Zero input size does not remove the fixed first stage.

## 5.5. P5: supply an actual Big-O witness

For $T(n) = 7n + 30$, find positive fixed $C$ and a threshold $n_0$ establishing $T(n) \in O(n)$. Is $O(n^2)$ also valid?

**Answer and explanation**

For $n \geq 30$, $30 \leq n$, so $7n + 30 \leq 8n$. Choose $C = 8$ and $n_0 = 30$. Also $T(n) \geq 7n$, so the growth is tightly linear. Since $8n \leq 8n^2$ for $n \geq 1$, the quadratic upper bound is valid but less informative.

## 5.6. P6: keep the dimensions explicit

An engineering sweep has $n$ diameter choices and $m$ speed choices. Every candidate requires $N_t$ fixed-cost time steps. State the selected step count. Then specialize it to $m = 4$, $N_t = 100$, both fixed. What changes when $m = n$ and $N_t = n$?

**Answer and explanation**

The selected simulation-step count is $nmN_t$. With four speeds and 100 fixed steps, it is $400n$, hence linear in $n$. If both additional dimensions equal $n$, it is $n^3$, hence cubic in $n$. The analysis changes because the problem's scaling assumptions change, not because loop syntax changes.

## 5.7. P7: compare pair definitions

For five readings, compare the number of checks in `maximum_gap()` with the count from `unique_pair_count(5)`. Are the two implementations doing precisely the same pair traversal? Do their growth orders differ?

**Answer and explanation**

The first visits all $5^2 = 25$ ordered pairs, including five self-pairs and both directions of each distinct pair. The second visits $5(5 - 1)/2 = 10$ unordered distinct-index pairs. The traversals and exact counts differ, but both have quadratic order for growing $n$.

## 5.8. P8: distinguish a floor from a ceiling

Trace `halving_steps(10)` and count body passes and condition tests. Separately, find the minimum number of exact interval halvings needed to reduce width 10 to at most 1. Explain why the answers differ.

**Answer and explanation**

Integer floor-halving gives $10 \to 5 \to 2 \to 1$: three body passes and four condition tests. Exact interval halving gives $10 \to 5 \to 2.5 \to 1.25 \to 0.625$: four halvings. The first uses $\lfloor \log_2 10 \rfloor = 3$ because of integer flooring; the second requires $\lceil \log_2 10 \rceil = 4$ to meet an upper-width requirement. Both have logarithmic growth but different exact rules.

## 5.9. P9: count recursive work rather than output magnitude

Compare three calculations at input 5: recursive factorial, exhaustive binary choices, and the naive counted Fibonacci function with $F(0) = F(1) = 1$. Give each output and the relevant source count.

**Answer and explanation**

Factorial returns 120 after five multiplications and six calls including the base call. Binary-choice traversal has 32 leaves and $2^6 - 1 = 63$ calls. Counted Fibonacci returns `(8, 15)`. These counts describe different structures and selected operations; the value 120 does not imply 120 factorial operations, and a numerical ratio of different counters is not a measured speedup.

## 5.10. P10: read a batch timing correctly

In an illustrative experiment, five batches each execute a task 20 times. The minimum observed batch duration is 0.040 s. What best observed per-call value should be reported? Does it establish the worst-case runtime or the complexity class?

**Answer and explanation**

The best observed batch-average duration is $0.040/20 = 0.002$ seconds per call, or 2 milliseconds. It does not establish a worst-case guarantee or a complexity class. The number is an explicitly hypothetical practice observation, not a row from the source timing data. Complexity needs an execution argument, and meaningful timing comparisons need repeated sizes and a disclosed protocol.

## 5.11. P11: preserve all requested prefix results

For `[2.0, -3.0, 4.0, 5.0]`, predict the complete prefix-total list. State the two selected element-visit counts for repeated summation and a running subtotal. What storage remains necessary after the improvement?

**Answer and explanation**

The output is `[2.0, -1.0, 3.0, 8.0]`. The repeated summations visit $1 + 2 + 3 + 4 = 10$ elements in total; the running method visits four. The requested four output totals must still be retained. Test output order and values, length, empty-input behavior, and non-mutation; use a justified numerical criterion for general floating-point inputs.

## 5.12. P12: judge the evidence in an efficiency report

A student writes: "My three-line function is $O(1)$ because it only calls one method. I tested one small dataset and it was faster, so it is always better. It returns only the final total instead of the requested full prefix list to save memory." Identify at least four separate problems.

**Answer and explanation**

Source length does not determine work. A method may hide a scan or copy. A single small timing observation does not establish an all-size ranking. The input size, case, and timed region are unstated. Returning a scalar changes the required output, so the claimed memory improvement is not a valid comparison for the original task. A repaired report must retain the contract and provide both correctness evidence and a justified growth argument before interpreting timings.

## 5.13. A final readiness check

You are ready to continue when you can explain, without memorized slogans, why a full scan is linear, a fixed-factor reduction is logarithmic, and a triangular pair traversal is quadratic. You should be able to select an input case, count a named operation, distinguish Big-O from a tight bound, and keep a measured duration separate from a mathematical model.

You should also be able to reject a misleading improvement that changes the output or discards necessary evidence. Faster code still needs correct units, appropriate data, clear contracts, independent checks, and honest reporting of limitations.

# 6 Quick Reference and Glossary

## 6.1. The compact analysis procedure

1. State the task, valid inputs, output, units, and accuracy requirement.
2. Define every growing size variable, including independent dimensions.
3. Declare the cost model and exactly which operation is being counted.
4. Trace ordinary and boundary cases; identify early exits and hidden calls.
5. Derive an exact selected count when possible, then identify its growth.
6. Distinguish the input case from the bound used to describe it.
7. State storage conventions: input, output, and temporary or active state.
8. Test behavior before and after an improvement; disclose the timing protocol and limitations.

| Source pattern | Selected count or statement |
|---|---|
| All-warning scan | $n$ threshold comparisons for every length-$n$ valid input. |
| First-warning scan | 1 best case and $n$ worst case for $n \geq 1$; $(n+1)/2$ only under the stated conditional uniform-position model. |
| Two consecutive scans | $2n$ visits, not $n^2$. |
| Three fixed checks per item | $3n$ actions. |
| Independent pair sweep | $nm$ considered pairs. |
| Unordered distinct-index pairs | $n(n-1)/2$. |
| Floor-halving loop | $\lfloor \log_2 n \rfloor$ body passes and one more condition test. |
| Exact bracket-width reduction | $\max(0, \lceil \log_2(L_0/\varepsilon_x) \rceil)$ minimum halvings. |
| Staged counting model | $n\lfloor \log_2 n \rfloor$ selected inner increments. |
| Square / cube traversal | $n^2$ / $n^3$ visits when $n$ is points per side. |
| All binary choices | $2^n$ leaves and $2^{n+1} - 1$ calls. |
| Recursive factorial | $n$ multiplications and $n+1$ calls under the unit-cost model. |
| Repeated prefix summations | $n(n+1)/2$ numerical element visits, with copying work as well. |
| Running prefix totals | $n$ numerical updates and $n$ retained output totals. |

## 6.2. Terminology

**Algorithmic work**
Executed actions described by a declared model, not necessarily literal CPU instructions.

**Input size**
A chosen measure of problem scale, such as list length, points per side, or representation length. The meaning must be stated.

**Cost model**
Assumptions about which actions have bounded cost and which operations are counted.

**Selected operation**
A named action, such as one threshold comparison, pair evaluation, numerical update, or function call.

**Exact count**
The exact number of occurrences of the selected operation for a specified size and input case.

**Order of growth**
How a cost behaves as the declared size becomes large, with constants and lower-order terms summarized.

**Dominant term**
A contribution that determines the long-run growth in a stated expression with fixed coefficients.

**Big-O**
An eventual upper-bound relation, $0 \leq T(n) \leq Cg(n)$ for all $n \geq n_0$ and fixed positive $C$.

**Tight order, Big-Theta**
Matching eventual upper and lower bounds up to fixed positive multipliers.

**Witness constants**
A fixed multiplier and starting threshold that establish a claimed Big-O inequality.

**Best case**
The smallest work over valid inputs of a given size.

**Worst case**
The largest work over valid inputs of a given size.

**Average case**
Expected work under an explicitly specified input distribution.

**Full scan**
Processing every entry of a collection.

**Early exit**
Stopping before every candidate is inspected because the required result is already determined.

**Hidden work**
Operations inside a compact expression, function, or method, such as scanning, copying, or membership checks.

**Consecutive stages**
Blocks executed one after another; their costs add.

**Independent nested dimensions**
Fully traversed combinations from separate ranges; their sizes multiply.

**Dependent bound**
A loop limit that changes with another index; total work generally needs a sum rather than a blind product.

**Logarithmic growth**
A pattern often created by fixed-factor reduction until a boundary is reached.

**Log-linear growth**
The $n \log n$ pattern, such as full-size work repeated over logarithmically many stages.

**Polynomial growth**
A fixed power such as $n^p$, with exponent $p$ independent of input size.

**Exponential growth**
A pattern such as visiting every configuration of $n$ binary choices, with $2^n$ leaves.

**Factorial growth**
In the optional example, the count of all orderings of $n$ distinct items; not the cost of merely calculating $n!$.

**Recursive call count**
Total invocations over the complete execution, including calls that have already returned.

**Active call depth**
The maximum number of simultaneously unfinished calls; different from total calls.

**Auxiliary storage**
Extra storage beyond explicitly excluded input or output. State the convention.

**Retained output**
Data that must remain available as the requested result, such as every prefix total.

**Elapsed duration**
A later timer reading minus an earlier reading for a specified measured region.

**Warm-up**
An unrecorded use of the same task before the recorded trials.

**Trial and batch**
A trial is one recorded timing observation; a batch may contain several task calls within that observation.

**Best observed timing**
The minimum recorded value under the protocol; not a worst-case guarantee.

**Median timing**
The middle value of the ordered observations; a descriptive summary, not an asymptotic proof.

**Scaling ratio**
A diagnostic such as $T(2n)/T(n)$, interpreted with a model and measurement limitations.

**Crossover**
The input size where two specified cost models are equal; dependent on those models and their constants.

**Amortized cost**
A sequence-level cost description that accounts for occasional expensive operations, such as list resizing.

**Unit-cost arithmetic**
A simplifying assumption that each selected arithmetic action has bounded cost; not an all-size model for growing integers.

**Non-mutation check**
A test that verifies the original input remains unchanged when the contract promises that behavior.

**Optimization**
A change intended to reduce a resource cost while preserving the stated task and its correctness requirements.

# A Source Basis and Slide Coverage

## A.1. What determines these notes

The requested basis is Yanwei Wang's supplied presentation, *Programming for Engineers: Week 10 — Understanding Program Efficiency I*, Fall 2026, 87 physical PDF pages. Its lecture sequence, contracts, examples, counter definitions, numerical values, and optional scope determine this document.

The earlier Week 10 LaTeX lecture bodies and surviving example files were used to resolve source wrapping and recover speaker-note qualifications, not to introduce a different syllabus. The original `timing.csv` and `timing_environment.json` were recovered from that same earlier package. Their values match the rounded source table. All measured values in the reading-note timing figure remain those of the source run.

### Source and addition boundaries

The comments in the numbered examples explain the displayed operations without silently introducing input-validation scans. Small execution calls added to source-only function or class excerpts are labelled. The benchmark-design solution, pressure exit-ticket implementation, prefix regression checks, and cumulative practice are worked additions. The smaller binary-choice tree is an explanatory diagram, not a different recurrence. New local timing outputs, retained only for verification, are not represented as the source benchmark.

One typesetting repair is explicit: the malformed product on source page 78 is written as $n \times n$, consistent with the all-pairs example and original speaker-note answer. The notes also preserve the source distinction between the floor-halving count and the ceiling for a width guarantee, the loose $O(2^n)$ Fibonacci bound and tight growth, and counts of different operation types.

### Further reading listed in the presentation

The following references are the reading alignment stated on source pages 85–86; they are not a claim that every chapter of those works has been reproduced here:

- *ENG 101 Course Specification*: Session 10; CLOs 1 and 2; Sessions 11–12 for continuation.
- Yanwei Wang, *Programming for Engineers with Python*: Sections 10.20–10.27; Appendix G.3–G.10 and G.32.
- John V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd edition: Chapter 9, particularly Sections 9.1–9.3.
- Python standard library: `time.perf_counter` and `timeit`. The source slide records a documentation check on 14 September 2026; that is the source's date, not a claim of a new check in these notes.

## A.2. Reproducibility and package contents

The LaTeX project contains the figures as editable TikZ/PGFPlots source, so no plotting package is needed to compile the document. The standalone source embeds the figure coordinates and code. The original timing CSV remains separately available for auditing. Python examples use only the standard library.

The verification scripts distinguish deterministic outputs from variable timing observations. Their checks support the displayed examples and identified test cases, not a proof of correctness for every possible program input. The numerical and operation-count arguments in the text state their assumptions explicitly.

## A.3. Map from every source page to these notes

Use physical PDF page numbers, not slide-footer numbers. Several adjacent slides may be developed together in one reading section. Cover, overview, divider, optional-reference, and closing pages are included so the mapping spans the complete source.

| PDF p. | Source topic | Reading location |
|---|---|---|
| 1 | Course title and metadata | About these notes, p. 2 |
| 2 | Week 10 at a glance | About these notes, p. 2 |
| 3 | Scope of the first efficiency unit | About these notes, p. 2 |
| 4 | Lecture 28 divider | Chapter 1, p. 6 |
| 5 | Lecture 28 learning goals | Chapter 1, p. 6 |
| 6 | Fewer lines, less work? | Section 1.1, p. 6 |
| 7 | Why scale changes engineering decisions | Section 1.1, p. 6 |
| 8 | Define the input size | Section 1.2, p. 7 |
| 9 | Time and memory | Section 1.3, p. 7 |
| 10 | Declare a simple cost model | Section 1.3, p. 7 |
| 11 | Count work and measure time | Section 1.3, p. 7 |
| 12 | Count threshold comparisons | Section 1.4, p. 8 |
| 13 | Trace warning and work counters | Section 1.4, p. 8 |
| 14 | Zero, one, and many inputs | Section 1.4, p. 8 |
| 15 | Early exit | Section 1.5, p. 9 |
| 16 | Same length, different paths | Section 1.5, p. 9 |
| 17 | An average requires an assumption | Section 1.6, p. 10 |
| 18 | One-line hidden scans | Section 1.7, p. 11 |
| 19 | Measure with perf_counter | Section 1.8, p. 11 |
| 20 | Repeat the experiment | Section 1.8, p. 11 |
| 21 | Fair comparison | Section 1.8, p. 11 |
| 22 | Recorded summation experiment | Section 1.9, p. 13 |
| 23 | Read the timing trend | Section 1.9, p. 13 |
| 24 | Scaling ratios | Section 1.10, p. 14 |
| 25 | Reproducible-measurement activity | Section 1.11, p. 14 |
| 26 | A method call can hide work | Section 1.12, p. 16 |
| 27 | Efficiency-claim checklist | Section 1.13, p. 16 |
| 28 | Lecture 28 exit ticket | Section 1.13, p. 16 |
| 29 | Lecture 29 divider | Chapter 2, p. 18 |
| 30 | Lecture 29 learning goals | Chapter 2, p. 18 |

| PDF p. | Source topic | Reading location |
|---|---|---|
| 31 | Dominant-term warm-up | Section 2.1, p. 18 |
| 32 | Three counted stages | Section 2.1, p. 18 |
| 33 | Keep the dominant term | Section 2.1, p. 18 |
| 34 | Growth is not a stopwatch | Section 2.1, p. 18 |
| 35 | Big-O definition | Section 2.2, p. 20 |
| 36 | A complete bound argument | Section 2.3, p. 20 |
| 37 | Graph of the eventual bound | Section 2.3, p. 20 |
| 38 | Big-O versus tight growth | Section 2.4, p. 21 |
| 39 | Case versus bound | Section 2.5, p. 22 |
| 40 | Simplification activity | Section 2.6, p. 22 |
| 41 | Consecutive stages add | Section 2.7, p. 23 |
| 42 | Nested independent ranges | Section 2.8, p. 23 |
| 43 | Separate dimensions | Section 2.8, p. 23 |
| 44 | Fixed inner bounds | Section 2.9, p. 24 |
| 45 | Dependent bounds need a sum | Section 2.10, p. 25 |
| 46 | Triangular pair grid | Section 2.10, p. 25 |
| 47 | Hidden membership scan | Section 2.11, p. 26 |
| 48 | Conditional routes | Section 2.12, p. 26 |
| 49 | Small-input ranking | Section 2.13, p. 27 |
| 50 | Crossover graph | Section 2.13, p. 27 |
| 51 | Logarithm bases | Section 2.14, p. 28 |
| 52 | Debugging explanations | Section 2.15, p. 28 |
| 53 | Lecture 29 exit ticket | Section 2.15, p. 28 |
| 54 | Lecture 30 divider | Chapter 3, p. 29 |
| 55 | Lecture 30 learning goals | Chapter 3, p. 29 |
| 56 | First map of growth classes | Section 3.1, p. 29 |
| 57 | Known-position retrieval | Section 3.2, p. 30 |
| 58 | Floor-halving counter | Section 3.3, p. 31 |
| 59 | Shrinking-size trace | Section 3.3, p. 31 |
| 60 | Interval-refinement cost | Section 3.4, p. 31 |
| 61 | Sampled-power energy | Section 3.5, p. 32 |
| 62 | Log-linear staged counting | Section 3.6, p. 33 |
| 63 | All-pairs maximum gap | Section 3.7, p. 34 |
| 64 | Three-dimensional grid | Section 3.8, p. 35 |
| 65 | Polynomial scaling | Section 3.8, p. 35 |
| 66 | Exhaustive binary choices | Section 3.9, p. 36 |
| 67 | Counted Fibonacci recursion | Section 3.10, p. 37 |
| 68 | Iterative Fibonacci | Section 3.10, p. 37 |
| 69 | Recursion is not a class | Section 3.11, p. 39 |
| 70 | Normalized growth table | Section 3.12, p. 39 |
| 71 | Normalized growth plot | Section 3.12, p. 39 |
| 72 | Declared-budget activity | Section 3.13, p. 40 |
| 73 | Time and retained storage | Section 3.14, p. 41 |
| 74 | Repeated prefix totals | Section 3.15, p. 41 |
| 75 | Running prefix totals | Section 3.15, p. 41 |
| 76 | Improve, test, measure again | Section 3.15, p. 41 |
| 77 | Misconception poll | Section 3.16, p. 43 |
| 78 | Lecture 30 exit ticket | Section 3.17, p. 44 |

| PDF p. | Source topic | Reading location |
|---|---|---|
| 79 | Final condition test | Section 4.1, p. 45 |
| 80 | Optional timeit helper | Section 4.2, p. 46 |
| 81 | Magnitude versus bit length | Section 4.3, p. 46 |
| 82 | Container-operation costs | Section 4.4, p. 47 |
| 83 | Optional factorial-growth contrast | Section 4.5, p. 47 |
| 84 | Practice before Week 11 | Section 4.6, p. 48 |
| 85 | Reading and source alignment | Section A.1, p. 57 |
| 86 | References and continuation | Section 4.7, p. 49 |
| 87 | Closing engineering message | Section 3.17, p. 44 |